R16868 Legged Systems

Problem Set #4: Human Locomotion System

Maria (mtagliaf)

November 1, 2026

Problem 1: Quick-release experiment with a Hill-type muscle model

Muscle parameters used throughout:

Parameter	Symbol	Value	Notes
Max. isometric force	F_{max}^{iso}	$6000\,\mathrm{N}$	
Optimum CE length	ℓ_{opt}	$8\,\mathrm{cm}$	
Force–length width	w	$0.56\,\ell_{opt}$	
Residual force at $\ell_{opt}(1 \pm w)$	c	0.05	lecture value
Max. contraction velocity	v_{max}	$12\,\ell_{opt}\,\mathrm{s}^{-1} = 0.96\,\mathrm{m/s}$	
Eccentric force enhancement	N_{ecc}	1.5	
Force–velocity curvature	K	5	lecture value

Sign convention: $v_{CE} < 0$ means the CE is *shortening* (concentric) and $v_{CE} > 0$ means it is *lengthening* (eccentric).

1.1 Force–length and force–velocity relations (2 pts)

Implementation. The active force–length relation is the bell-shaped curve

$$f_l^a(\ell_{CE}) = \exp\left(\ln(c)\,\left|\frac{\ell_{CE} - \ell_{opt}}{w\,\ell_{opt}}\right|^3\right), \tag{1}$$

which equals 1 at $\ell_{CE} = \ell_{opt}$, drops to $c = 0.05$ at $\ell_{CE} = \ell_{opt}(1 \pm w)$, and is essentially zero at $\ell_{CE} = 0$ and $\ell_{CE} = 2\ell_{opt}$ (no passive elasticity is included). The Hill-type force–velocity relation is

$$f_v(v_{CE}) = \begin{cases} \dfrac{v_{max} + v_{CE}}{v_{max} - K\,v_{CE}}, & v_{CE} \le 0 \quad \text{(concentric)} \\ N_{ecc} + (N_{ecc} - 1)\,\dfrac{v_{CE} - v_{max}}{7.56\,K\,v_{CE} + v_{max}}, & v_{CE} > 0 \quad \text{(eccentric)} \end{cases} \tag{2}$$

It equals 1 in isometric conditions, falls to 0 at $v_{CE} = -v_{max}$, and rises steeply towards $N_{ecc} = 1.5$ for lengthening. Both relations are implemented as MATLAB functions (`f_length`, `f_velocity`) in `p1_1_force_surface.m` (Appendix A). Figure 1 shows the two relations, and Figure 2 shows the product

$$M(\ell_{CE}, v_{CE}) = f_l^a(\ell_{CE})\,f_v(v_{CE}) \tag{3}$$

for $\ell_{CE} \in [0, 2\ell_{opt}]$ and $v_{CE} \in [-v_{max}, +v_{max}]$.

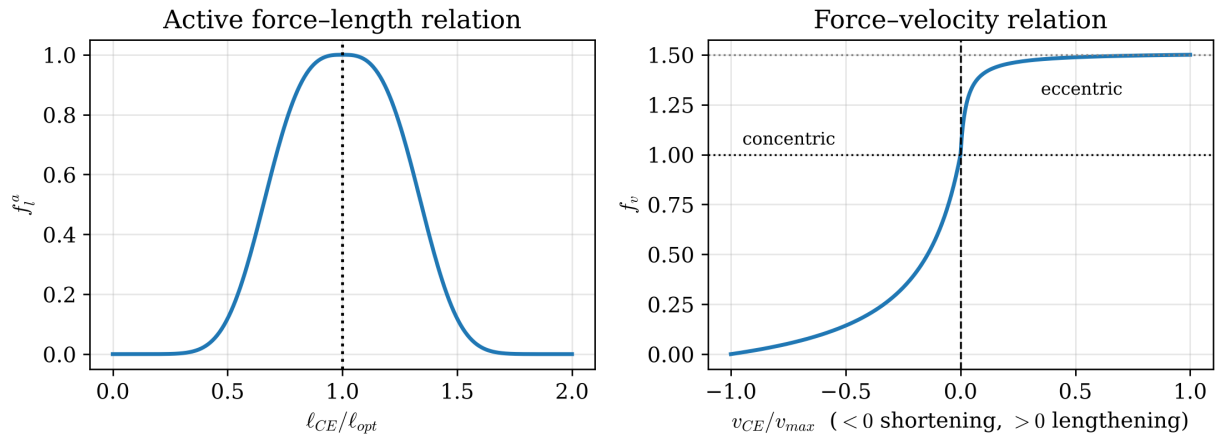
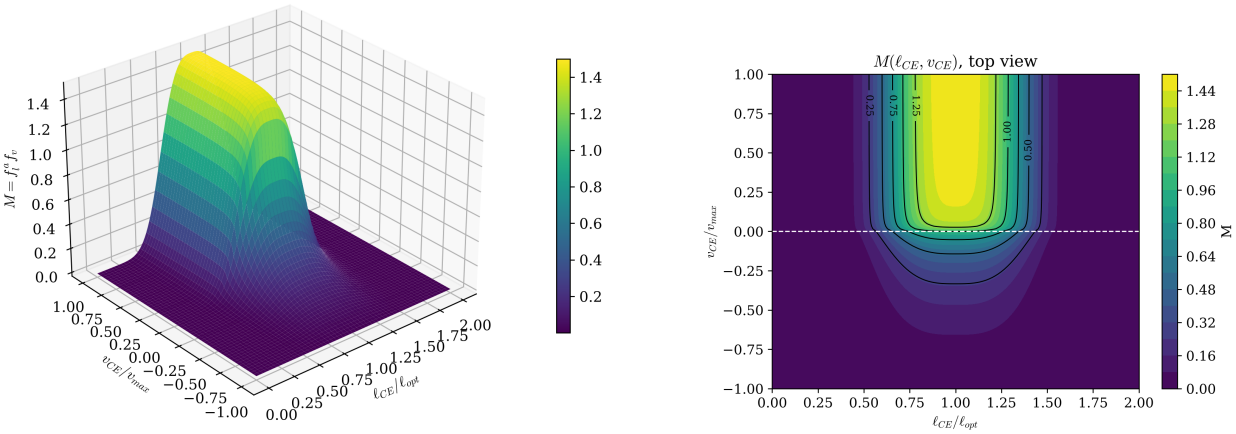


Figure 1: Active force–length relation $f_l^a(\ell_{CE})$ (left) and force–velocity relation $f_v(v_{CE})$ (right).



(a) Surface plot

(b) Top view with iso-lines

Figure 2: Normalized CE force capacity $M(\ell_{CE}, v_{CE}) = f_l^a f_v$. The dashed line in (b) marks $v_{CE} = 0$ (isometric); below it the CE shortens, above it the CE lengthens.

Observations. M is a ridge centered on $\ell_{CE} = \ell_{opt}$ whose height increases monotonically with v_{CE}: it goes to zero for fast shortening, equals 1 in isometric conditions, and saturates near 1.5 for lengthening. For a given activation, the muscle can therefore produce far more force while it is being stretched than while it shortens.

Why we feel warm hiking uphill but cool off on the way down. The CE force is $F_{CE} = F_{max}^{iso}\, A\, M(\ell_{CE}, v_{CE})$, so the activation needed to produce a force F is

$$A = \frac{F}{F_{max}^{iso}\, M(\ell_{CE}, v_{CE})}. \tag{4}$$

The large antigravity muscles of the legs (vasti, gluteus, soleus/gastrocnemius) must support roughly body weight in both directions, so the required force F is about the same. What changes is the direction in which these muscles move:

- **Uphill:** the extensors *shorten* under load (concentric, $v_{CE} < 0$) and do positive work to lift the body. On this side of the surface $M < 1$ and drops quickly with speed: at $|v_{CE}| = 0.3\, v_{max}$, $M \approx 0.28$. To produce the same force, the nervous system must raise A and recruit many more fibers. Every active fiber consumes ATP for cross-bridge cycling and calcium pumping. Muscle converts only about 25% of this metabolic energy into mechanical work, so most of it is released as heat and we warm up.
- **Downhill:** the same muscles *lengthen* under load (eccentric, $v_{CE} > 0$) and act as brakes, absorbing energy. Here $M > 1$ ($M \approx 1.47$ at $0.3\, v_{max}$), so the same force needs about $1.47/0.28 \approx 5$ times *less* activation. Far fewer fibers are active, ATP use is low, and metabolic heat production drops. This matches the classic finding that negative work costs only a fraction of the metabolic energy of the same amount of positive work [1].

In short, the surface shows that force per unit activation is much higher during lengthening than during shortening. Descending is therefore metabolically cheap, and climbing is expensive and generates heat. (The mechanical energy absorbed during descent also ends up as heat, but part of it is dissipated outside the muscle fibers, e.g. in tendons, footwear and the ground. Overall heat production is still much lower than uphill.)

1.2 Excitation–contraction coupling (1 pt)

Model. The excitation–contraction coupling is a first-order low-pass filter whose time constant depends on whether the muscle is being switched on or off:

$$\dot{A} = \frac{S - A}{\tau}, \qquad \tau = \begin{cases} \tau_R = 30\,\text{ms}, & S > A \quad \text{(rising: activation)} \\ \tau_F = 80\,\text{ms}, & S < A \quad \text{(falling: deactivation)} \end{cases} \qquad S(t) = 0.5 \sin(2\pi t) + 0.5. \tag{5}$$

We switch on the sign of $S - A$ (stimulation above or below the current activation). This is the standard activation/deactivation rule, and it also works for the step input used in part 1.4. Switching on the sign of $\dot{S}$ instead gives almost the same result here (peak and trough of A differ by less than 0.03). The muscle starts at rest, $A(0) = 0$.

Figure 3 shows the Simulink model. It is built and simulated for 1 s by `p1_2_ecc.m` (Appendix A), which creates `p1_2_ecc.slx`.

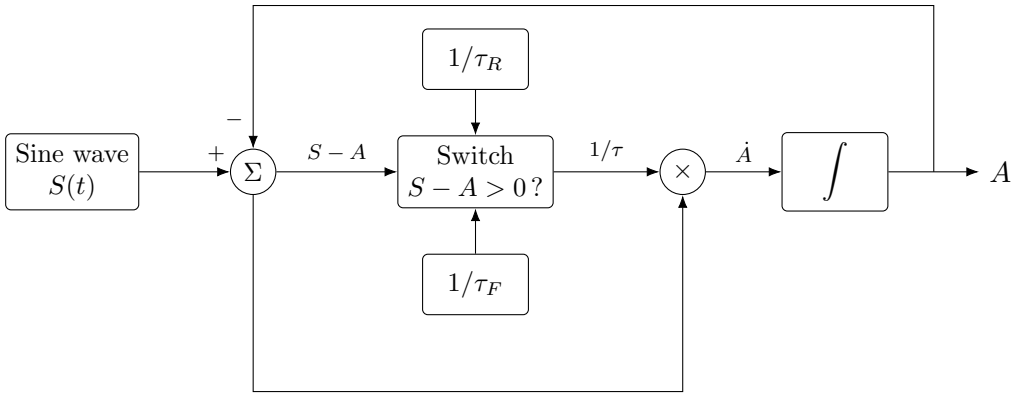


Figure 3: Simulink model of the excitation–contraction coupling (Eq. 5).

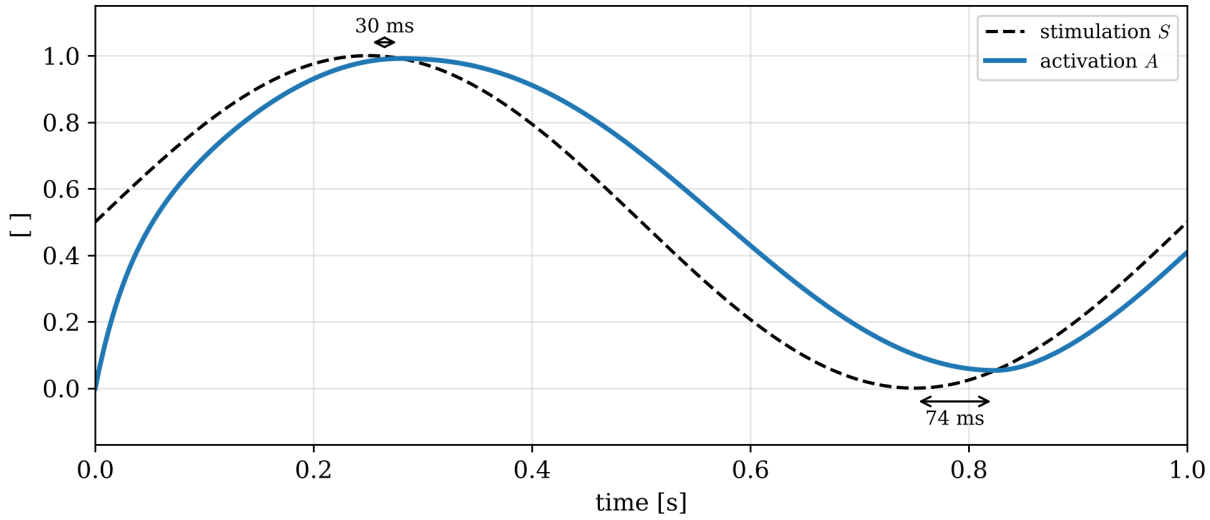


Figure 4: Neural stimulation S and muscle activation A over 1 s. The arrows mark the delay between the extremes of S and A.

Discussion. A is a smoothed, delayed copy of S (Figure 4). The muscle cannot turn on or off instantly:

- **Start-up:** A starts at 0 while S starts at 0.5, so A first has to catch up. This start-up effect is gone after about 0.1 s ($\approx 3\tau_R$).
- **Delay:** A always lags behind S. It peaks 30 ms after S peaks and bottoms out 74 ms after S bottoms out.
- **On fast, off slow:** because $\tau_F > \tau_R$, the muscle switches on faster than it switches off. The delay after S falls is more than twice the delay after S rises. Physiologically, activation is the fast release of calcium in the muscle fiber, and deactivation is the slower pumping of calcium back into storage.
- **Never fully relaxed:** A reaches 0.99 at the top but only drops to 0.05 at the bottom, because

S starts rising again before A has had time to decay. As a result, A stays high for longer than S and its average (≈ 0.55) is higher than the average of S (0.5).

In short, the muscle responds to the nerve signal with a delay, and it takes longer to relax than to activate. At 1 Hz the effect is moderate, but at faster stimulation the delay and smoothing would become larger, so A could no longer follow S. This is one reason the nervous system must send commands ahead of time for fast movements.

1.3 Contractile element model (1 pt)

Model. The complete contractile element combines parts 1.1 and 1.2 into one block:

$$F_{CE} = F_{max}^{iso}\, f_l^a(\ell_{CE})\, f_v(v_{CE})\, A, \tag{6}$$

where A comes from the excitation–contraction coupling (Eq. 5) and f_l^a, f_v are the relations from Eqs. 1 and 2. The CE is built as a Simulink subsystem with three inputs (S, ℓ_{CE}, v_{CE}) and two outputs (F_{CE}, A), see Figure 5. Inside, ℓ_{CE} and v_{CE} are first normalized by ℓ_{opt} and v_{max}, then passed to MATLAB Function blocks that call the lecture functions `CE_force_length_relation` and `CE_force_velocity_relation`. The products are multiplied by A and scaled by F_{max}^{iso}. The subsystem is created by `build_CE.m` so it can be reused in parts 1.4–1.6; `p1_3_ce.m` builds the test model `p1_3_ce.slx` (Appendix A).

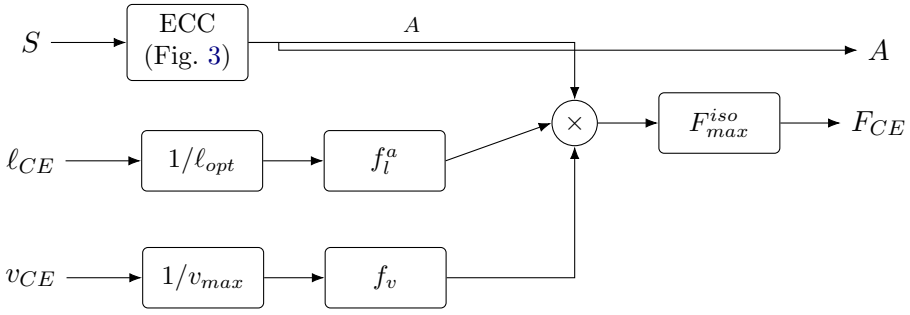


Figure 5: CE subsystem: $F_{CE} = F_{max}^{iso} f_l^a f_v A$ (Eq. 6).

Test. To check the block, S is the sine wave from part 1.2 while ℓ_{CE} and v_{CE} are held constant (Figure 6):

- **Isometric at** ℓ_{opt} ($f_l^a = f_v = 1$): $F_{CE} = F_{max}^{iso} A$, so the force is just the activation scaled to 6000 N. The peak is 5950 N because A peaks at 0.99.
- $\ell_{CE} = 1.2\,\ell_{opt}$, $v_{CE} = -0.2\,v_{max}$: the muscle is longer than optimal ($f_l^a = 0.87$) and shortening ($f_v = 0.40$), so every force value is scaled by $0.87 \times 0.40 = 0.35$ and the peak drops to 2080 N.

Both match the hand-computed values, so the block is wired correctly. The force keeps the same delayed shape as A; the length and velocity only change how much of F_{max}^{iso} is available.

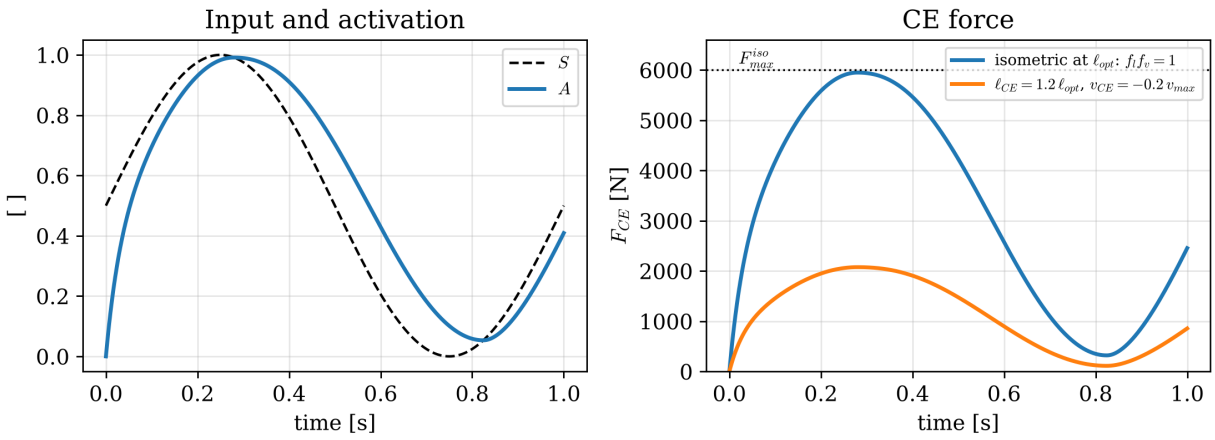


Figure 6: CE test: activation (left) and resulting force (right) for the two constant (ℓ_{CE}, v_{CE}) settings.

1.4 Quick-release experiment, $m = 400\,\mathrm{kg}$ (2 pts)

Setup. The CE of part 1.3 hangs from the ceiling with the mass below it (Figure 1), so $l_m = \ell_{CE}$ and $v_m = v_{CE}$ (positive when the mass moves down and the muscle lengthens). Newton's law for the mass gives

$$m\,\ddot{l}_m = mg - F_m, \qquad F_m = F_{CE}(l_m, \dot{l}_m, A). \tag{7}$$

In Simulink (Figure 7) a Switch block sets the acceleration to zero while $t < t_q$ (muscle locked at ℓ_{opt}) and to $g - F_m/m$ afterwards; two integrators then give v_m and l_m, which are fed back into the CE. The stimulation is $S = 1$ throughout. The model is built and run by `run_quick_release.m`, called from `p1_4_quick_release.m` (Appendix A).

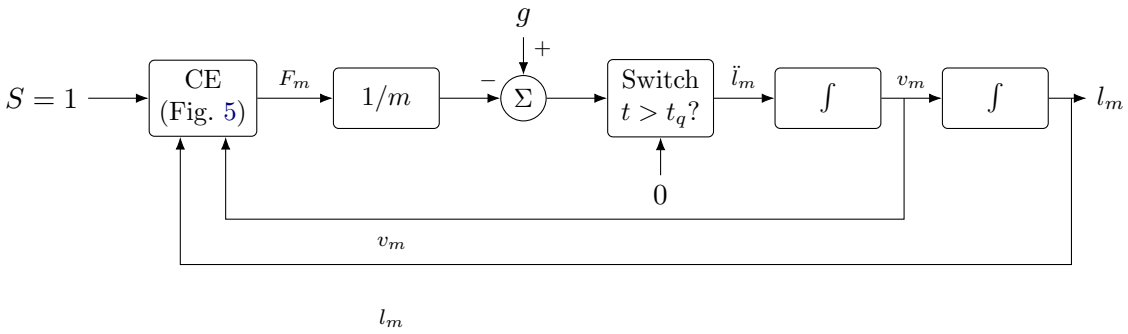


Figure 7: Simulink model of the quick-release experiment: the CE pulls on the mass, and the mass motion sets the CE length and velocity.

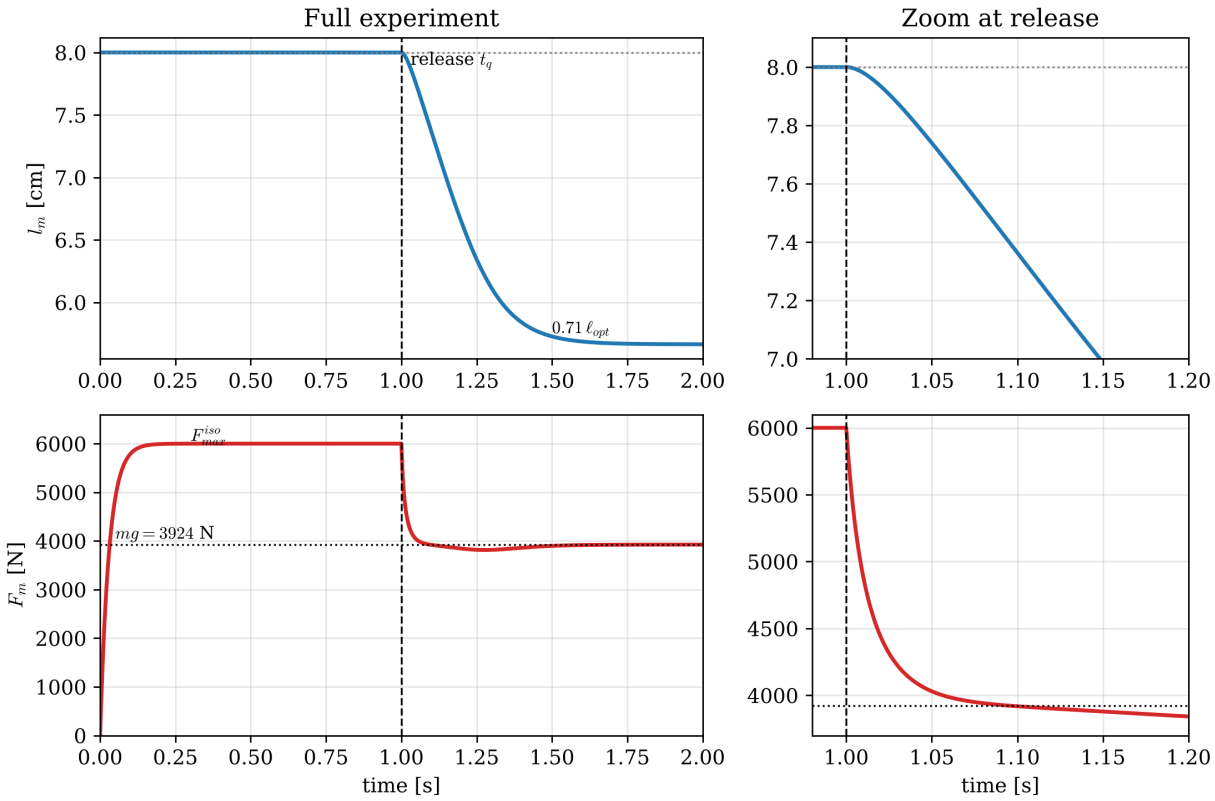


Figure 8: Simulated quick release with $m = 400\,\text{kg}$: muscle length (top) and force (bottom). Left: whole experiment; right: the first 200 ms after release.

What the simulation shows.

- **Before release ($t < t_q$):** the muscle is isometric at ℓ_{opt}, so $F_m = F_{max}^{iso} A$. Force rises with the activation time constant and reaches the full 6000 N after about 0.15 s.
- **At release:** the muscle pulls with 6000 N against a weight of only $mg = 3924\,\text{N}$, so the mass accelerates upward and the muscle starts to shorten. As soon as it shortens, the force-velocity relation cuts the force down: within about 50 ms the force drops to $\approx mg$ and the shortening velocity settles at $0.076\,\text{m/s}$ ($0.08\,v_{max}$). This is exactly the velocity at which $f_v = mg/F_{max}^{iso} = 0.65$, i.e. the muscle shortens at the speed where its force balances the load. The length trace is smooth; there is no jump at release.
- **Later:** as the muscle gets shorter, f_l^a decreases, so it can lift the load less and less quickly. Shortening slows down and stops at $l_m = 0.71\,\ell_{opt} = 5.7\,\text{cm}$, where the isometric force $F_{max}^{iso} f_l^a$ equals mg. The force settles at mg with a small undershoot around $t = 1.3\,\text{s}$.

Comparison with real quick-release experiments. Jewell and Wilkie [2] released tetanized frog muscle from isometric contraction to a constant (isotonic) load, and Civan and Podolsky [3] did the same on single fibres with fast force clamps. Their length records show three phases:

1. an *immediate* shortening step at release, within 1–2 ms, which is the elastic recoil of the series elastic element (tendon and cross-bridges) as the force drops from P_0 to the load;

2. a short transient (tens of ms) in which the shortening velocity is not yet steady; Civan and Podolsky show it can even oscillate, and they attribute it to cross-bridge kinetics;

3. steady isotonic shortening at the velocity predicted by the force-velocity curve for that load.

The force itself drops to the load almost instantly, because the lever used in these experiments is very light.

The model reproduces phase 3: the shortening velocity it settles at is exactly the one given by f_v, and the slowing down as the muscle leaves its optimum length is also seen in long isotonic records. It does *not* reproduce phase 1 (there is no elastic element, so there is no length step) or phase 2 (the Hill-type CE has no cross-bridge dynamics, so the velocity simply rises smoothly). Also, the force in the simulation takes about 50 ms to come down rather than falling instantly, because the 400 kg mass has to be accelerated first. So the product model mirrors the steady-state behaviour of real muscle in quick release, but not the fast transients right after release. The missing elastic step is what part 1.6 adds with the series element.

1.5 Mass sweep and force–velocity relation (2 pts)

Method. The experiment of part 1.4 is repeated for $m = 1, 10, 50, 100, 200, 400, 600, 700, 800$ kg (`p1_5_mass_sweep.m`, Appendix A). The slope of $l_m(t)$ right after release is read off the velocity trace $v_m = \dot{l}_m$ (Figure 9a): after a short transient in which the mass is accelerated, v_m reaches a clear extremum (for shortening) or plateau (for lengthening), and that value is taken as the slope. Each point is marked with a dot in the figure.

What to expect. Right after release the muscle is still at ℓ_{opt} ($f_l^a = 1$) and fully active ($A = 1$). Once the mass has stopped accelerating, the muscle force balances the weight:

$$mg = F_{max}^{iso}\, f_v(\dot{l}_m) \qquad \Longrightarrow \qquad m(\dot{l}_m) = \frac{F_{max}^{iso}}{g}\, f_v(\dot{l}_m). \tag{8}$$

So the plot of m against the initial slope should be nothing but the force-velocity curve, rescaled by $F_{max}^{iso}/g = 612$ kg. Masses lighter than 612 kg are lifted (negative slope, concentric branch); heavier masses pull the muscle out (positive slope, eccentric branch). This is also how the force-velocity relation is measured in real quick-release experiments: one load per trial, one velocity per load.

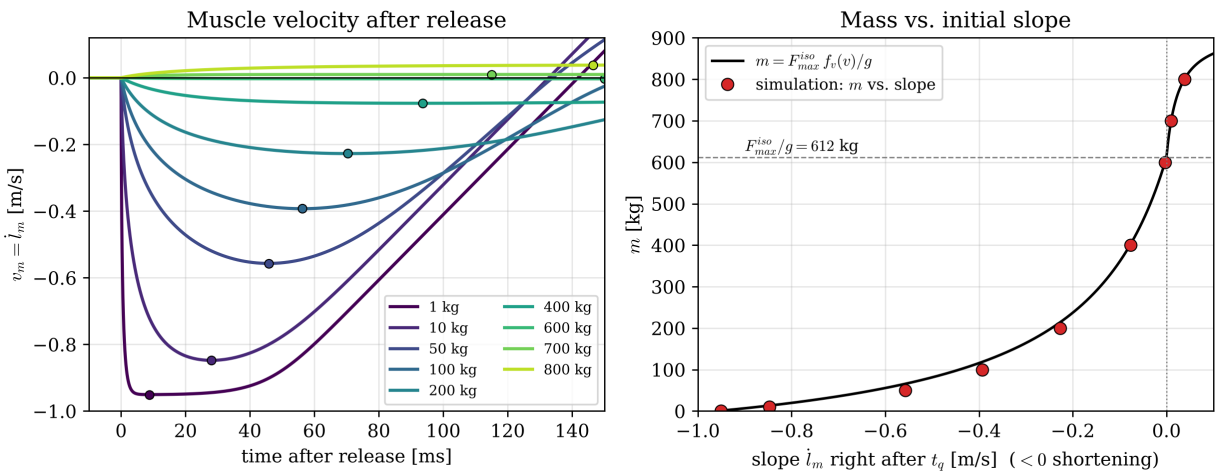


Figure 9: (a) Muscle velocity after release for all masses; dots mark the value taken as the slope. (b) Mass against initial slope (dots) compared with the force-velocity relation of the model, Eq. 8 (line).

m [kg]	slope $\dot{l}_m$ [m/s]	$\dot{l}_m / v_{max}$	t after release [ms]	$\dot{l}_m$ from f_v alone [m/s]
1	−0.951	−0.99	9	−0.951
10	−0.848	−0.88	28	−0.873
50	−0.557	−0.58	46	−0.626
100	−0.393	−0.41	56	−0.442
200	−0.227	−0.24	70	−0.245
400	−0.076	−0.08	94	−0.078
600	−0.003	−0.00	150	−0.003
700	+0.010	+0.01	115	+0.010
800	+0.038	+0.04	147	+0.038

Table 1: Measured initial slope for each mass and the velocity at which $f_v = mg/F^{iso}_{max}$.

What we find.

- The points follow the force-velocity curve closely (Figure 9b). The heavy end (400–800 kg) and the very light end (1 kg) match f_v to within 1%: the quick-release experiment recovers the force-velocity relation that was put into the model. The 1 kg mass is almost a free shortening and gives $\dot{l}_m \approx -v_{max}$; the 600 kg mass sits right at F^{iso}_{max}/g and barely moves.

- For the middle masses (10–200 kg) the measured slope is 5–11% *slower* than f_v predicts. The reason is visible in Figure 9a: these masses take 30–70 ms to accelerate, and by the time the velocity peaks the muscle has already shortened to 0.76–0.94 ℓ_{opt}, where $f^a_l < 1$. The muscle is then weaker than at ℓ_{opt} and cannot reach the full f_v speed. (Correcting with the actual f^a_l at the time of the extremum puts these points back on the curve.)

- This is the same difficulty experimenters face: the "instantaneous" velocity after release is only well defined if the load is accelerated much faster than the muscle changes length, which is why real quick-release setups use light isotonic levers rather than heavy hanging masses.

In short, m against initial slope reproduces the force-velocity relation of the CE, as it should, with small deviations in the mid range caused by the inertia of the mass and the force-length relation.

1.6 Option: series elasticity (2 pts)

Model. A series elastic element (tendon) is put in series with the CE, so $l_m = \ell_{CE} + \ell_{SE}$. The SE force is the lecture relation

$$F_{SE} = F_{max}^{iso} \left(\frac{\ell_{SE}/\ell_{slack} - 1}{\epsilon_{ref}} \right)^2 \quad (\ell_{SE} > \ell_{slack}), \qquad \ell_{slack} = 0.23\,\text{m},\ \epsilon_{ref} = 0.04, \tag{9}$$

i.e. the tendon reaches F_{max}^{iso} at 4% strain (9.2 mm). Since the junction between CE and SE is massless, both carry the same force, $F_m = F_{SE} = F_{CE}$. This is used the same way as in the lecture model `m01`: the SE stretch $\ell_{SE} = l_m - \ell_{CE}$ gives the force, and the CE velocity then follows from the *inverted* force-velocity relation,

$$f_v = \frac{F_m}{F_{max}^{iso}\, f_l^a(\ell_{CE})\, A} \quad \Longrightarrow \quad \dot{\ell}_{CE} = v_{max}\, f_v^{-1}(f_v), \qquad \ell_{CE} = \int \dot{\ell}_{CE}\, dt. \tag{10}$$

The muscle is locked at $l_m = \ell_{opt} + \ell_{slack}$ (tendon just slack, CE at ℓ_{opt}). The subsystem is built by `build_MTU.m`, the experiment by `run_quick_release_SE.m`, and the plots by `p1_6_series_elasticity.m` (Appendix A).

Quick release with $m = 400\,\text{kg}$. Figure 10 overlays the CE-only result of part 1.4 with the CE+SE result (length shown as change from the locked length, since the two muscles have different total lengths).

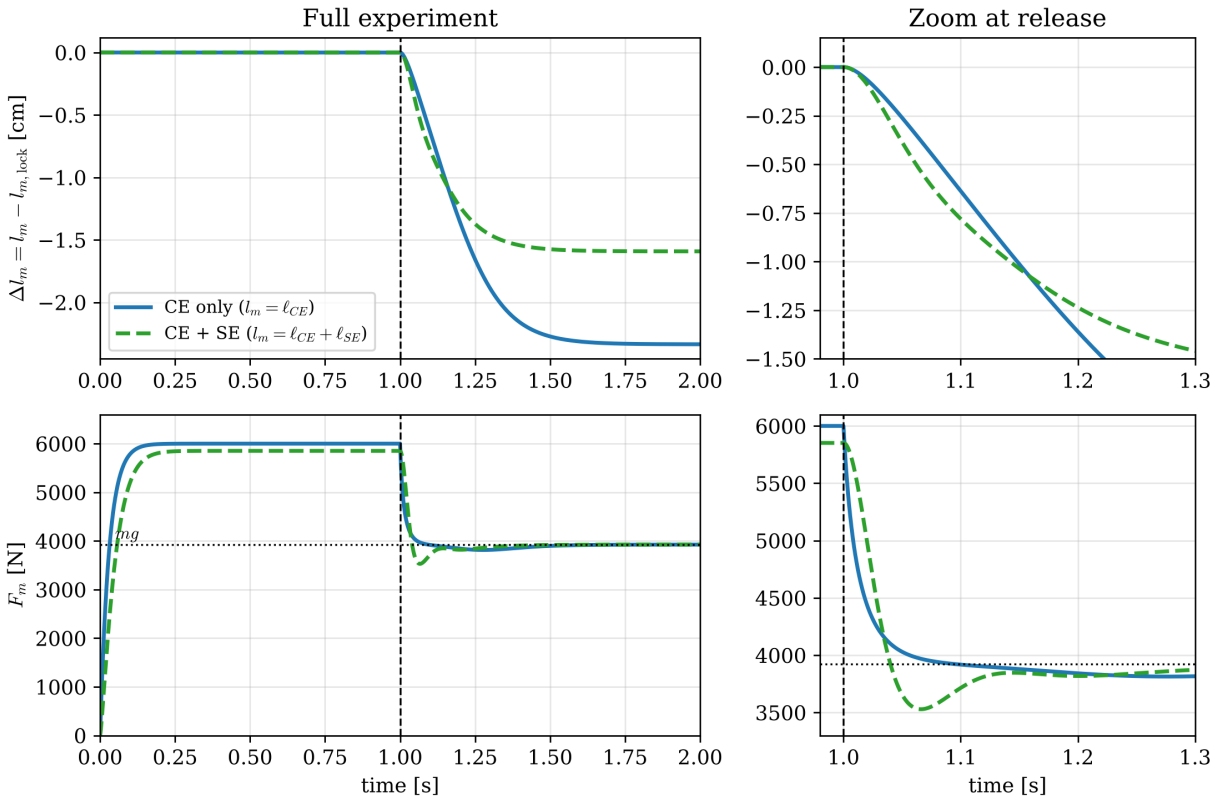


Figure 10: Quick release with $m = 400\,\mathrm{kg}$, with and without series elasticity. Top: change of muscle length from its locked value; bottom: muscle force. Right column: zoom on the first 300 ms after release.

- **Same:** the overall picture is unchanged. Force builds up before release, drops to mg after release, the muscle shortens and finally settles where the isometric CE force equals the weight. The CE ends at $0.71\,\ell_{opt}$ in both cases.
- **Before release:** with the SE, the isometric force is 5850 N instead of 6000 N. Even though l_m is fixed, the CE shortens by 9 mm to stretch the tendon, ends up at $0.89\,\ell_{opt}$ and so has $f_l^a = 0.98$. This internal shortening is real: in an isometric contraction the fibres shorten and the tendon stretches.
- **At release:** the force now drops *more slowly* at first (at 10 ms: 5590 N with SE vs. 4880 N without), because the tendon has to unload before the force can fall. The tendon then recoils, pushing the length change ahead of the CE-only curve: Δl_m is -0.5 cm at 60 ms vs. -0.3 cm without SE, and the peak muscle velocity rises from -0.076 to -0.105 m/s.
- **Oscillation:** the tendon and the mass form a spring-mass system (stiffness $\approx 1.3 \times 10^6$ N/m, period ≈ 110 ms) that is damped only by the CE's force-velocity relation. The force therefore undershoots to 3530 N (10% below mg) at 66 ms and rings once before settling; without the SE it approached mg monotonically.
- **Total shortening:** Δl_m ends at -1.6 cm instead of -2.3 cm, because the CE had already shortened 9 mm before release and the tendon stays stretched by 7.4 mm under the load.

So the SE adds exactly the missing feature from part 1.4: a fast elastic recoil at release. With a light load it would show up as the immediate length step of the experiments; with a 400 kg mass it shows up as a damped oscillation.

Mass sweep with SE. Figure 11 repeats the sweep of part 1.5 with the SE. Two slopes are read off for each mass: the slope of l_m (what an experimenter measures) and the slope of ℓ_{CE} alone (what the force-velocity relation is about).

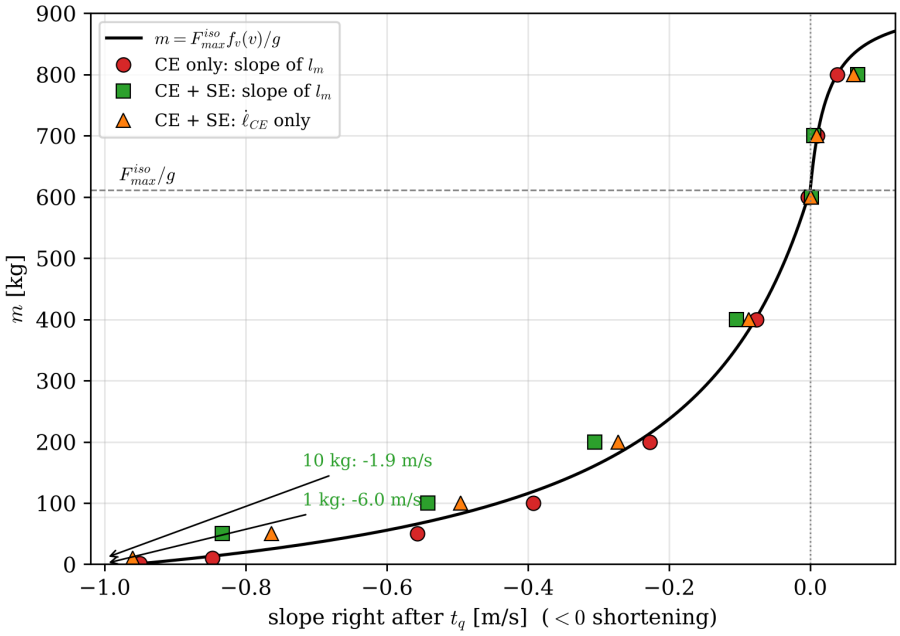


Figure 11: Mass against initial slope for the CE-only model (circles), for the CE+SE model using the slope of l_m (squares) and of ℓ_{CE} (triangles), compared with the force-velocity relation (line). The 1 kg and 10 kg squares are off the scale on the left.

m [kg]	CE only	CE+SE: $\dot{l}_m$	CE+SE: $\dot{\ell}_{CE}$	from f_v
1	−0.951	−6.03	−0.960	−0.951
10	−0.848	−1.93	−0.960	−0.873
50	−0.557	−0.834	−0.764	−0.626
100	−0.393	−0.542	−0.496	−0.442
200	−0.227	−0.306	−0.272	−0.245
400	−0.076	−0.105	−0.087	−0.078
600	−0.003	+0.001	+0.000	−0.003
700	+0.010	+0.005	+0.009	+0.010
800	+0.038	+0.067	+0.062	+0.038

Table 2: Initial slopes [m/s] with and without series elasticity.

What changes.

- **The slope of l_m no longer gives the force-velocity relation.** With the SE, the squares lie well to the left of the f_v curve: the apparent shortening velocity is 30–40% too fast for 100–400 kg, and for light masses it is absurd (1.9 m/s for 10 kg, 6 m/s for 1 kg, far above $v_{max} = 0.96$ m/s). The reason is that the stretched tendon stores energy (≈ 50 J at F^{iso}_{max}) and catapults a light mass when released. The initial slope of l_m then measures tendon recoil, not muscle shortening. A force-velocity curve read from these slopes would have no v_{max} at all.
- **The CE velocity is closer, but still not on the curve.** The triangles are faster than f_v by 10–20%, for two reasons: the CE starts at $0.89\,\ell_{opt}$ rather than ℓ_{opt}, so $f^a_l < 1$ and it must move faster to carry the load; and during the recoil the tendon briefly unloads the CE (for 1 and 10 kg it goes fully slack, so the CE shortens at $-v_{max}$ regardless of the mass).
- **Heavy masses:** above F^{iso}_{max}/g the tendon stretches further as the mass pulls, so the lengthening slopes are also larger (800 kg: 0.067 vs. 0.038 m/s).

In short, adding the series elasticity makes the simulated quick release look more like the real experiment (elastic step at release), but for the same reason the naive "slope right after t_q" now reflects the tendon rather than the contractile element. To recover f_v one has to wait for the recoil to finish and measure the steady shortening velocity, which is exactly what Jewell and Wilkie did when they separated the series elastic from the contractile component [2].

Problem 2: Central pattern generator for biped gaits

Baseline parameters throughout: $x_0 = 5$, $\tau_x = 1$, $\tau_a = 12$, $w' = 2.5$.

2.1 Single neuron (2 pts)

Model. The left extensor neuron is a Matsuoka neuron with self-adaptation:

$$y_{el} = \max(x_{el}, 0), \qquad \dot{x}_{el} = \frac{1}{\tau_x}\Big(x_0 + \sum_i w_i y_i - w' x' - x_{el}\Big), \qquad \dot{x}' = \frac{1}{\tau_a}\,(y_{el} - x'), \tag{11}$$

with $\sum_i w_i y_i = 0$ for now. Both state equations have the form $\tau \dot{z} = (\text{inputs}) - z$, i.e. each is a first-order low-pass filter with a leak, exactly like the activation dynamics of part 1.2. In Simulink each one is therefore the same three blocks, Sum $\rightarrow$ Gain $1/\tau$ $\rightarrow$ Integrator, with the integrator output fed back into the Sum (Figure 12). The output nonlinearity $\max(x, 0)$ is a Saturation block with lower limit 0 and no upper limit. The neuron is packaged as a subsystem with inputs x_0 and $\sum_i w_i y_i$ and outputs y, x, x' (`build_neuron.m`), so that the same block can be copied for the flexor and for the right leg in parts 2.2 and 2.3. `p2_1_neuron.m` builds the one-neuron model `p2_1_neuron.slx`, simulates 40 s and runs the parameter sweeps (Appendix A).

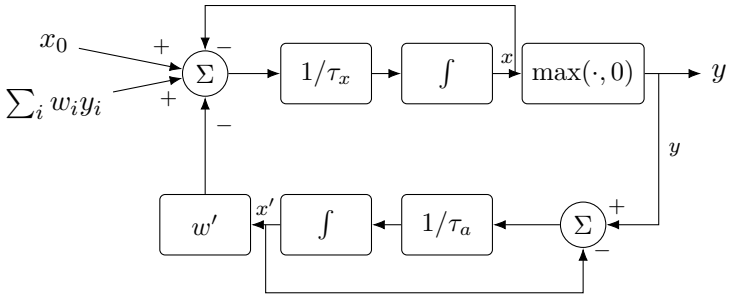


Figure 12: Simulink subsystem of one Matsuoka neuron (Eq. 11): membrane loop on top, adaptation loop below, coupled through the weight w'.

What to expect. When $x > 0$ the neuron is a linear second-order system. Its steady state follows from setting the derivatives to zero: $x' = y = x$ and $x = x_0 - w'x$, so

$$y_{ss} = \frac{x_0}{1 + w'} = \frac{5}{3.5} = 1.43. \tag{12}$$

Its characteristic equation is $s^2 + (1/\tau_x + 1/\tau_a)\, s + (1 + w')/(\tau_x \tau_a) = 0$. With the baseline values the roots are $s = -0.50$ and -0.58: real and negative, so the response is just barely non-oscillatory and dies out with a time constant of about 2 s. The roots become complex (damped ringing) when the discriminant is negative, which for $\tau_x = 1$, $w' = 2.5$ is the range $0.08 < \tau_a < 11.9$; the baseline $\tau_a = 12$ sits right at the edge. The real part is always negative, so a single neuron can never oscillate on its own.

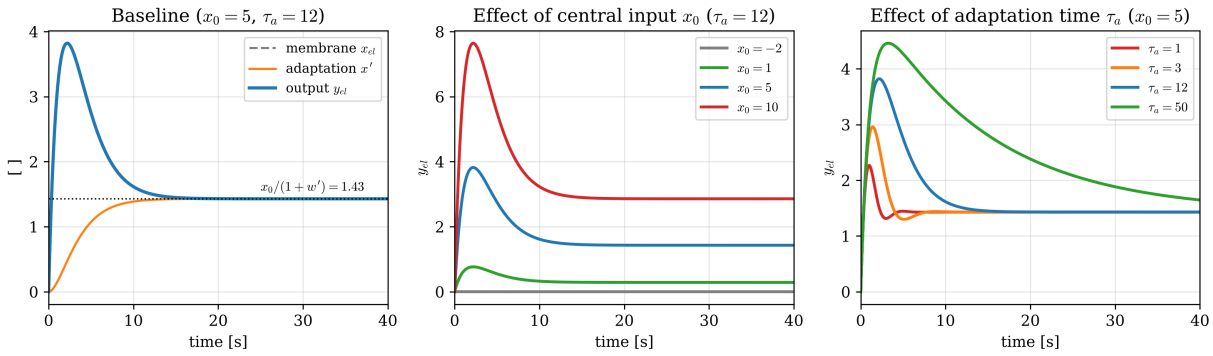


Figure 13: Single neuron. Left: baseline time course of membrane potential, adaptation and output (membrane and output coincide because $x > 0$ throughout). Middle: varying the central input x_0. Right: varying the adaptation time constant τ_a.

Results (Figure 13).

- **Baseline:** the output shoots up towards x_0 with the fast membrane time constant, peaks at 3.8 after 2.2 s, and then decays as the adaptation x' slowly builds up and subtracts $w'x'$ from the drive. It settles at 1.43 as predicted. This is the classic "adapting" response of a sensory or motor neuron: a strong initial burst followed by a lower sustained rate.
- **x_0 sets the size, not the shape.** The output scales linearly with x_0 (peak 0.76/3.8/7.6 and plateau 0.29/1.43/2.86 for $x_0 = 1/5/10$) while the timing is identical. For $x_0 \leq 0$ the membrane

potential never becomes positive and the neuron stays silent. So x_0 acts as a gain or "volume" control on the neuron's activity, and as an on/off switch.

- τ_a **sets the time course.** A fast adaptation ($\tau_a = 1$) cuts the burst short (peak 2.3 at 1 s) and the output rings once before settling. A slow adaptation ($\tau_a = 50$) lets the burst grow larger (peak 4.5 at 3.3 s) and the decay to the plateau takes much longer than 40 s. The plateau itself does not depend on τ_a. So τ_a controls how long the neuron can keep firing strongly before it fatigues, which will set the period of the oscillator in part 2.2.

2.2 Extensor–flexor oscillator (2 pts)

Model. Two copies of the neuron block are connected so that each one's output inhibits the other (Figure 14): the extensor receives $w_{fe}\, y_f$ and the flexor receives $w_{ef}\, y_e$, with $w_{fe} = w_{ef} = -1.5$. The weights are Gain blocks on the wires *between* the neurons, feeding the $\sum_i w_i y_i$ port; nothing inside the neuron block changes. Both neurons get the same tonic drive $x_0 = 5$. The extensor starts at $x_{el}(0) = 1$ and the flexor at 0; without this asymmetry the two identical neurons would stay exactly equal forever. `p2_2_oscillator.m` builds `p2_2_oscillator.slx` and runs all the sweeps below (Appendix A).

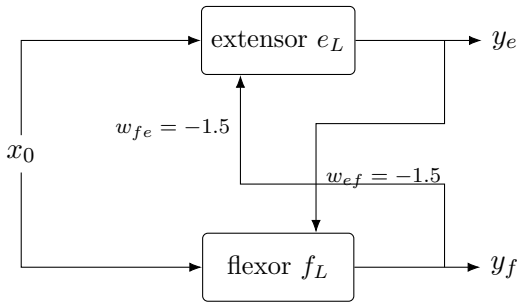


Figure 14: Half-center oscillator: two neuron blocks with mutual inhibition through external weight blocks.

Why it oscillates, and when it stops. The single neuron of part 2.1 only adapts; the pair also *competes*. Whichever neuron is more active inhibits the other, but then tires through its own adaptation x', its inhibition weakens, and the other neuron takes over. This is Brown's half-center mechanism, and it needs both ingredients: inhibition to make one neuron win, adaptation to make it lose again.

The question of when this works can be answered by linearizing around the symmetric state in which both neurons are equally active. Writing $d = x_e - x_f$ and $d' = x'_e - x'_f$ for the difference (antiphase) mode, Eq. 11 gives

$$\tau_x \dot{d} = (|w| - 1)\, d - w' d', \qquad \tau_a \dot{d'} = d - d', \tag{13}$$

where $|w| = 1.5$ is the inhibition strength. The symmetric state becomes unstable (the trace of this 2×2 system turns positive) when

$$\frac{|w| - 1}{\tau_x} > \frac{1}{\tau_a} \quad \Longleftrightarrow \quad \tau_a > \frac{\tau_x}{|w| - 1} = 2\tau_x \quad (\text{for } |w| = 1.5). \tag{14}$$

So with $w = -1.5$ the pair oscillates exactly when $\tau_a > 2\tau_x$, which is the condition the problem asks about: the adaptation must be slow enough compared with the membrane for the winning

neuron to hold on for a while before it tires. The same formula also shows that the inhibition must exceed $|w| = 1$ for any τ_a (we checked: $w = -1$ never oscillates, $w = -1.2$ does).

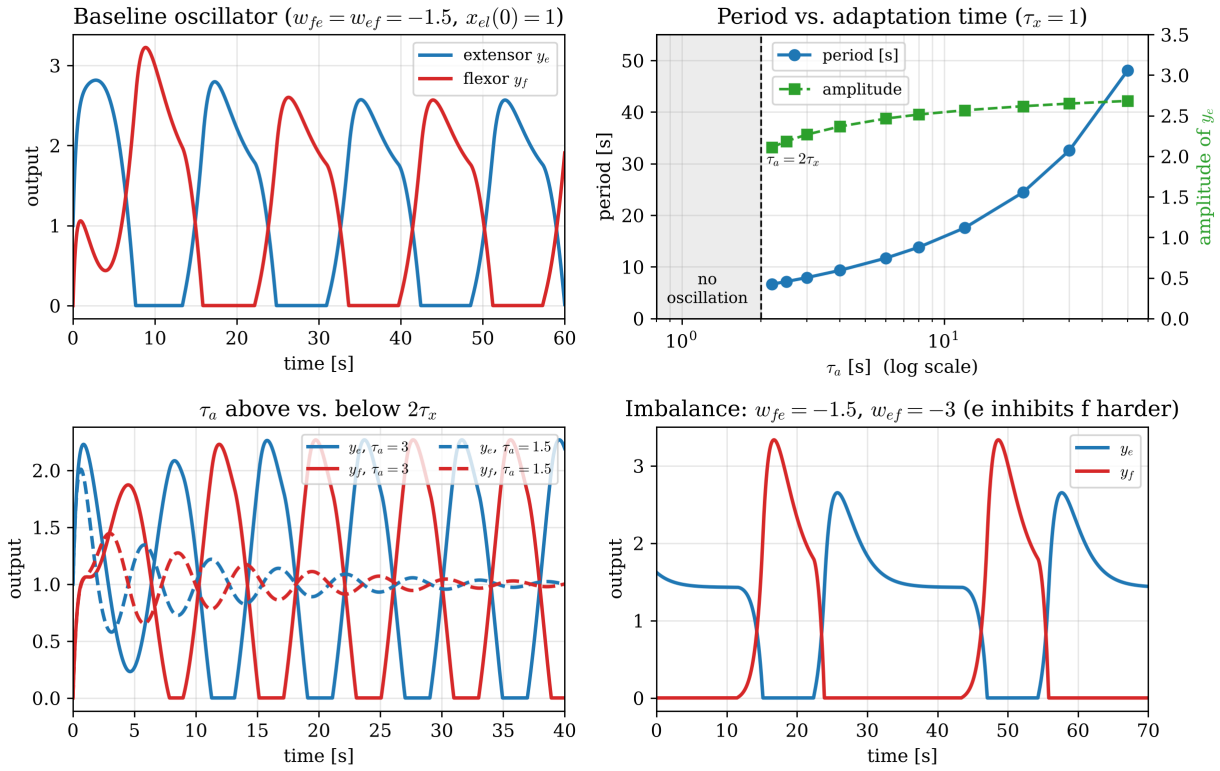


Figure 15: (a) Baseline oscillator. (b) Period and amplitude against τ_a; no oscillation for $\tau_a \leq 2\tau_x$. (c) Time traces for $\tau_a = 3$ (sustained) and $\tau_a = 1.5$ (decays to a steady state). (d) Unequal weights: the extensor inhibits the flexor twice as hard.

Results (Figure 15).

- **Baseline:** after one transient cycle the pair settles into a stable antiphase rhythm with period 17.6 s and peak output 2.6. Each neuron is active for about 65% of the period, so the bursts overlap a little: $w = -1.5$ is not strong enough to silence the other neuron completely.
- τ_a **sets the period** (panel b): it grows almost in proportion to τ_a (7 s at $\tau_a = 2.5$, 17.6 s at 12, 48 s at 50), while the amplitude changes little (2.1–2.7). Adaptation is the clock of the oscillator.
- $\tau_a < 2\tau_x$ **kills the oscillation** (panels b, c). At $\tau_a = 1.5$ the two outputs ring against each other with shrinking swings and settle at the symmetric steady state $y_e = y_f = x_0/(1 + w' + |w|) = 1.0$. The adaptation is now as fast as the membrane, so a neuron tires before it has managed to suppress its partner, and no neuron can win. At $\tau_a = 3$, just above the threshold, the rhythm is sustained but fast (7.9 s). Note that this threshold moves with the weights: stronger inhibition lowers it, $\tau_a > \tau_x/(|w| - 1)$.
- **Unequal weights change the duty cycle** (panel d). With $w_{ef} = -3$ (extensor inhibits flexor twice as hard) the extensor is the dominant neuron 72% of the time and the flexor only gets short bursts, though taller ones (3.3 vs. 2.7) because it has rested longer. The period also stretches to

32 s. Going the other way, $w_{ef} = -1$ lets the flexor dominate (active 85% of the period), and at $w_{ef} = -0.5$ the oscillation dies with the flexor permanently on. So an imbalance biases which neuron is on for longer, and a large imbalance ends in a steady winner. This is how a CPG could produce the long stance (extensor) and short swing (flexor) phases of walking.

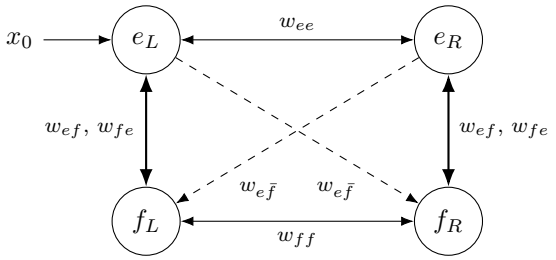
2.3 Coupled oscillators and gait patterns (3 pts)

Model. Four copies of the neuron block (e_L, f_L, e_R, f_R) are connected all-to-all as in Figure 2 of the assignment. All connections are collected in a 4×4 weight matrix W, where W_{ij} is the weight from neuron j into neuron i; each entry is a Gain block on the corresponding wire, and the three incoming wires of each neuron are added in a Sum block that feeds its $\sum_i w_i y_i$ port (`p2_3_gaits.m`, Appendix A). The network is built once; a gait is then just a choice of W and of the initial membrane potentials. All neurons receive the same drive $x_0 = 5$. Following the assignment, a leg is taken to push on the ground whenever its extensor neuron is active ($y_e > 0$).

The connections fall into four groups, each with a clear job (Figure 16):

- **Within a leg**, w_{ef} (e→f) and w_{fe} (f→e): the half-center of part 2.2. It makes the leg's rhythm, and its *imbalance* sets how long the extensor is on (stance fraction): e-dominant for a long stance, f-dominant for a short one.
- **Extensor to extensor**, w_{ee} ($e_L \leftrightarrow e_R$): mutual inhibition pushes the two extensors *apart* in phase, i.e. it forces alternation.
- **Flexor to flexor**, w_{ff}: same effect, helps to keep the legs alternating.
- **Extensor to the other leg's flexor**, $w_{e\bar{f}}$ ($e_L \to f_R$, $e_R \to f_L$): when the left extensor is on it silences the right flexor, which frees the right extensor. This pulls the two extensors *together*, i.e. it forces synchrony.

So the recipe is: use the within-leg imbalance to set stance duration, and choose between w_{ee} (alternate) and $w_{e\bar{f}}$ (synchronize) to set the phase between the legs. Matsuoka's 1987 analysis of frequency and pattern control in such networks [5] is the basis for this reasoning.



solid: half-centers and homologous coupling (alternation)
dashed: crossed e→f coupling (synchrony)

Figure 16: The four-neuron network and the role of each connection group. Every neuron receives x_0 (shown for one).

gait	w_{ef}	w_{fe}	w_{ee}	w_{ff}	$w_{e\bar{f}}$	initial x (e_L, f_L, e_R, f_R)
walk	-2.0	-1.0	-0.5	-0.5	0	$(1, 0, 0, 0)$
run	-1.0	-2.5	-1.0	0	0	$(1, 0, 0, 0)$
hop	-1.0	-2.5	0	0	-0.5	$(1, 0, 0, 0)$

Table 3: Synaptic weights that reproduce the extensor patterns of Figure 2. Baseline $x_0 = 5$, $\tau_x = 1$, $\tau_a = 12$, $w' = 2.5$ for all neurons.

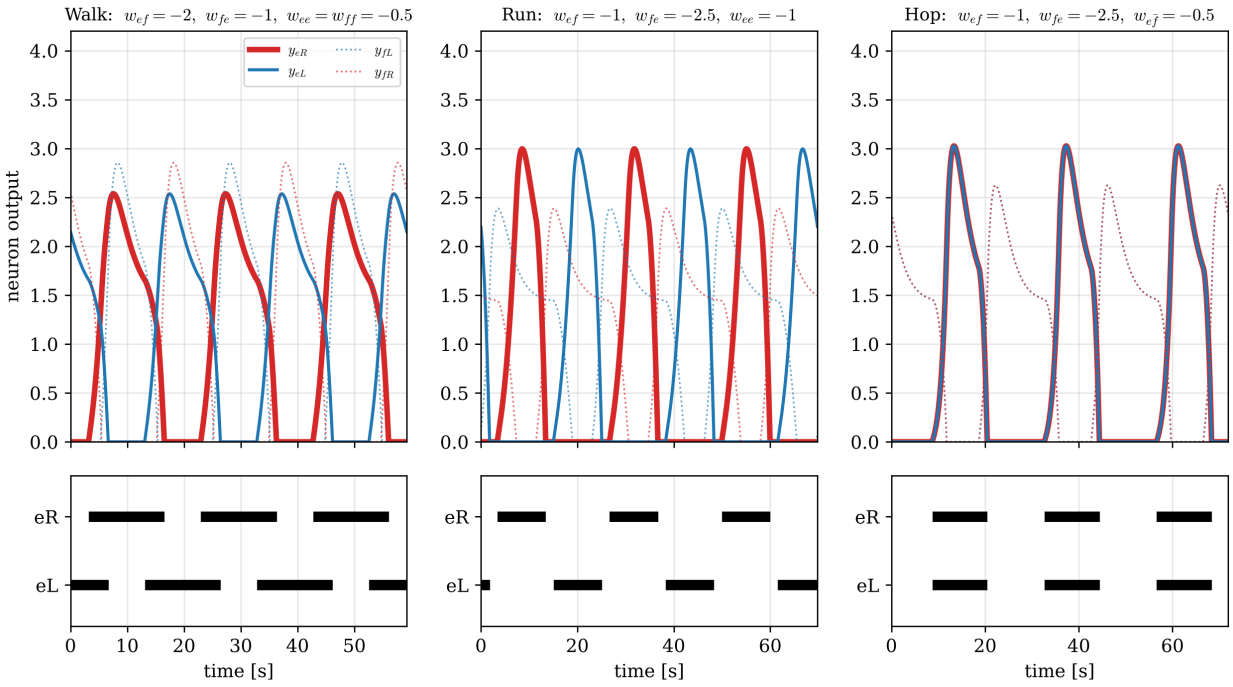


Figure 17: Three gait patterns from the same network with different weights (three periods after the transient). Top: extensor outputs (bold) and flexor outputs (dotted). Bottom: extensor activity bars in the style of Figure 2.

Results (Table 3, Figure 17).

- **Walking:** extensor-dominant legs plus e–e and f–f inhibition. The extensors alternate exactly (phase 0.5), each is active for 68% of the period, and the bursts overlap for 35% of the cycle: double support, with the legs never both off the ground. This is the walking pattern of Figure 2.
- **Running:** flexor-dominant legs plus stronger e–e inhibition. Still antiphase, but each extensor is on for only 43% of the period and there are gaps of 13% of the cycle in which *neither* extensor is active: flight phases. The bursts are taller (3.0 vs. 2.5) because the extensor has rested longer, in line with the higher ground forces of running.
- **Hopping:** the same flexor-dominant legs, but the crossed $e \to \bar{f}$ inhibition instead of e–e. The two extensors now fire together (phase 0), 49% on, with 53% of the cycle in flight. This pattern is reached from any initial condition we tried, including the asymmetric one used for the other gaits: the crossed coupling makes synchrony the only stable pattern.

What we learned from exploring.

- **Initial membrane potentials matter.** With the walking weights, starting all four neurons at the same potential leaves the network stuck at a steady state (the symmetric solution of part 2.2); $x_{eL}(0) = 1$ breaks the tie and the antiphase rhythm appears. With the running weights, a symmetric start $(1, 0, 1, 0)$ instead produces a weak in-phase (hopping-like) rhythm. So the same weights can hold more than one pattern, and the initial state picks one. A real CPG would use a brief asymmetric command or sensory input to do this.
- **Weights are not unique.** Many settings give each pattern; what matters is the sign structure. Without any cross-leg coupling the two legs run independently and keep whatever phase they started with.
- **Time scale.** With $\tau_a = 12$ the periods are 20–24 s, far from a 1 s stride. The equations only depend on time through τ_x and τ_a, so scaling both by the same factor scales the period by that factor without changing the pattern ($\tau_x = 0.05$, $\tau_a = 0.6$ gives a stride of about 1 s).

Answer to the research question. Yes: two coupled Matsuoka oscillators can produce extensor activity patterns that match walking, running and hopping. Stance duration is set by the within-leg weight imbalance and inter-leg phase by which cross-leg connections are present (homologous inhibition for alternation, crossed inhibition for synchrony), while x_0 scales amplitude and τ_a sets the period. The CPG alone does not, however, know anything about the body: it cannot adapt the timing to the terrain or to a stumble. That is what the reflex pathways in a model like the neuromuscular walker of the course provide, and real locomotion control is thought to combine both.

References

[1] B. C. Abbott, B. Bigland, and J. M. Ritchie, "The physiological cost of negative work," *The Journal of Physiology*, vol. 117, no. 3, pp. 380–390, 1952.

[2] B. R. Jewell and D. R. Wilkie, "An analysis of the mechanical components in frog's striated muscle," *The Journal of Physiology*, vol. 143, no. 3, pp. 515–540, 1958.

[3] M. M. Civan and R. J. Podolsky, "Contraction kinetics of striated muscle fibres following quick changes in load," *The Journal of Physiology*, vol. 184, no. 3, pp. 511–534, 1966.

[4] K. Matsuoka, "Sustained oscillations generated by mutually inhibiting neurons with adaptation," *Biological Cybernetics*, vol. 52, pp. 367–376, 1985.

[5] K. Matsuoka, "Mechanisms of frequency and pattern control in the neural rhythm generators," *Biological Cybernetics*, vol. 56, pp. 345–353, 1987.

A MATLAB code

Problem 1.1: `p1_1_force_surface.m`

```
%% Problem 1.1 - CE force-length and force-velocity relations
% Surface plot of M(lCE, vCE) = f_l(lCE) * f_v(vCE)
% for lCE in [0, 2*lopt] and vCE in [-vmax, +vmax].
%
% Sign convention: vCE < 0 -> shortening (concentric)
%                  vCE > 0 -> lengthening (eccentric)

clear; close all; clc;

%% Muscle parameters (from assignment)
mp.Fmax = 6000;          % [N]   max isometric force
mp.lopt = 0.08;          % [m]   optimum CE length (8 cm)
mp.w    = 0.56;          % [lopt] width of force-length curve (w = 0.56*lopt)
mp.c    = 0.05;          % []    remaining force at lCE = lopt*(1 +/- w)
mp.vmax = 12 * mp.lopt;  % [m/s] max contraction velocity (12 lopt/s)
mp.N    = 1.5;           % []    eccentric force enhancement (Necc)
mp.K    = 5;             % []    curvature (shape) factor of f_v

%% Grid
nL = 101; nV = 101;
lCE = linspace(0, 2*mp.lopt, nL);          % [m]
vCE = linspace(-mp.vmax, mp.vmax, nV);     % [m/s]
[L, V] = meshgrid(lCE, vCE);

%% Evaluate relations
FL = f_length(L, mp);
FV = f_velocity(V, mp);
M  = FL .* FV;

%% Plots: individual relations
figure('Name','CE relations','Position',[100 100 900 350]);
subplot(1,2,1);
plot(lCE/mp.lopt, f_length(lCE, mp), 'LineWidth', 2); grid on;
xlabel('l_{CE} / l_{opt}'); ylabel('f_l^a');
title('Active force-length relation');

subplot(1,2,2);
plot(vCE/mp.vmax, f_velocity(vCE, mp), 'LineWidth', 2); grid on; hold on;
xline(0, '--k'); yline(1, ':k');
xlabel('v_{CE} / v_{max}  (<0 shortening, >0 lengthening)'); ylabel('f_v');
title('Force-velocity relation');

%% Surface plot of M = f_l * f_v
figure('Name','M(lCE, vCE)');
surf(L/mp.lopt, V/mp.vmax, M, 'EdgeColor', 'none'); hold on;
contour3(L/mp.lopt, V/mp.vmax, M, 15, 'k');
colormap(parula); colorbar;
xlabel('l_{CE} / l_{opt}');
ylabel('v_{CE} / v_{max}');
zlabel('M = f_l^a \cdot f_v');
title('Normalized CE force capacity M(l_{CE}, v_{CE})');
view(-135, 30); grid on;

% Example hiking cases at l_CE = l_opt, |v| = 0.3 vmax (printed below)
vh = 0.3 * mp.vmax;
Mup   = f_length(mp.lopt, mp) * f_velocity(-vh, mp);  % uphill: shortening
Mdown = f_length(mp.lopt, mp) * f_velocity( vh, mp);  % downhill: lengthening

```

```
%% Top view (contour map) of M
figure('Name','M contour');
contourf(L/mp.lopt, V/mp.vmax, M, 20, 'LineColor', 'none'); hold on;
contour(L/mp.lopt, V/mp.vmax, M, [0.25 0.5 0.75 1 1.25], 'k', 'ShowText', 'on');
yline(0, '--w');
colormap(parula); colorbar;
xlabel('l_{CE} / l_{opt}'); ylabel('v_{CE} / v_{max}');
title('M(l_{CE}, v_{CE}), top view');

%% Export figures for the Overleaf write-up (figures/ folder)
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(findobj('Name','CE relations'), 'figures/fl_fv.png', 'Resolution', 300);
exportgraphics(findobj('Name','M(lCE, vCE)'),  'figures/M_surface.png', 'Resolution', 300);
exportgraphics(findobj('Name','M contour'),    'figures/M_contour.png', 'Resolution', 300);

fprintf('At l_CE = l_opt, |v_CE| = 0.3 v_max:\n');
fprintf('  uphill   (shortening): M = %.3f -> activation needed for F: A = F/(Fmax*%.3f)\n', Mup,
    Mup);
fprintf('  downhill (lengthening): M = %.3f -> activation needed for F: A = F/(Fmax*%.3f)\n', Mdown
    , Mdown);
fprintf('  ratio of required activation (up/down) = %.1f\n', Mdown/Mup);

%% ---------- Local functions ----------
function fl = f_length(lCE, mp)
% Active force-length relation (bell-shaped, no passive elasticity)
%   f_l = exp( ln(c) * | (lCE - lopt) / (w*lopt) |^3 )
fl = exp(log(mp.c) .* abs((lCE - mp.lopt) ./ (mp.w * mp.lopt)).^3);
end

function fv = f_velocity(vCE, mp)
% Hill-type force-velocity relation
%   concentric (v <= 0): f_v = (vmax + v) / (vmax - K v)
%   eccentric  (v >  0): f_v = N + (N-1) (v - vmax) / (7.56 K v + vmax)
fv = zeros(size(vCE));
con = vCE <= 0;
ecc = ~con;
fv(con) = (mp.vmax + vCE(con)) ./ (mp.vmax - mp.K .* vCE(con));
fv(ecc) = mp.N + (mp.N - 1) .* (vCE(ecc) - mp.vmax) ./ (7.56 * mp.K .* vCE(ecc) + mp.vmax);
end
```

Problem 1.2: p1_2_ecc.m

```
%% Problem 1.2 - Excitation-contraction coupling in Simulink
% Builds the Simulink model p1_2_ecc.slx programmatically, simulates 1 s,
% and plots the neural stimulation S(t) and muscle activation A(t).
%
% Model:  dA/dt = (S - A) / tau,   tau = tau_R  if S > A  (activation, S "rising")
%                                  tau = tau_F  if S < A  (deactivation, S "falling")
%         S(t)  = 0.5 sin(2*pi*t) + 0.5
%
% Run this file as a script (it uses the base workspace variable mp).

clear; close all; clc;

%% Parameters
mp.tauR = 0.030;   % [s] rise (activation) time constant
mp.tauF = 0.080;   % [s] fall (deactivation) time constant
```

```
mp.A0   = 0;       % [ ] initial activation (muscle at rest)
Tsim    = 1;       % [s] simulation time

%% Build the Simulink model
mdl = 'p1_2_ecc';
if bdIsLoaded(mdl), close_system(mdl, 0); end
if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
new_system(mdl);
open_system(mdl);

% Neural stimulation S = 0.5 sin(2 pi t) + 0.5
add_block('simulink/Sources/Sine Wave', [mdl '/Stimulation S'], ...
    'SineType', 'Time based', 'Amplitude', '0.5', 'Bias', '0.5', ...
    'Frequency', '2*pi', 'Phase', '0', 'Position', [40 100 80 140]);

% Error S - A
add_block('simulink/Math Operations/Sum', [mdl '/S - A'], ...
    'Inputs', '+-', 'Position', [150 105 180 135]);

% Choose 1/tau: 1/tauR if (S - A) > 0, otherwise 1/tauF
add_block('simulink/Sources/Constant', [mdl '/1 over tauR'], ...
    'Value', '1/mp.tauR', 'Position', [150 30 210 60]);
add_block('simulink/Sources/Constant', [mdl '/1 over tauF'], ...
    'Value', '1/mp.tauF', 'Position', [150 190 210 220]);
add_block('simulink/Signal Routing/Switch', [mdl '/rising or falling'], ...
    'Criteria', 'u2 > Threshold', 'Threshold', '0', ...
    'Position', [260 85 300 155]);

% dA/dt = (S - A) / tau
add_block('simulink/Math Operations/Product', [mdl '/times 1 over tau'], ...
    'Inputs', '2', 'Position', [350 100 380 160]);

% Integrator -> A
add_block('simulink/Continuous/Integrator', [mdl '/Integrator'], ...
    'InitialCondition', 'mp.A0', 'Position', [430 115 460 145]);

% Outputs: scope + workspace logging
add_block('simulink/Sinks/Scope', [mdl '/Scope S and A'], ...
    'Position', [560 40 590 90]);
try
    set_param([mdl '/Scope S and A'], 'NumInputPorts', '2');
catch
    set_param([mdl '/Scope S and A'], 'NumInputPorts', 2);
end
add_block('simulink/Sinks/To Workspace', [mdl '/S_log'], ...
    'VariableName', 'S_log', 'SaveFormat', 'Timeseries', ...
    'Position', [560 180 620 200]);
add_block('simulink/Sinks/To Workspace', [mdl '/A_log'], ...
    'VariableName', 'A_log', 'SaveFormat', 'Timeseries', ...
    'Position', [560 230 620 250]);

% Wiring
add_line(mdl, 'Stimulation S/1',      'S - A/1',             'autorouting', 'on');
add_line(mdl, 'S - A/1',              'rising or falling/2', 'autorouting', 'on');
add_line(mdl, 'S - A/1',              'times 1 over tau/1',  'autorouting', 'on');
add_line(mdl, '1 over tauR/1',        'rising or falling/1', 'autorouting', 'on');
add_line(mdl, '1 over tauF/1',        'rising or falling/3', 'autorouting', 'on');
add_line(mdl, 'rising or falling/1',  'times 1 over tau/2',  'autorouting', 'on');
add_line(mdl, 'times 1 over tau/1',   'Integrator/1',        'autorouting', 'on');
```

```
add_line(mdl, 'Integrator/1',         'S - A/2',             'autorouting', 'on');  % feedback
add_line(mdl, 'Integrator/1',         'Scope S and A/2',     'autorouting', 'on');
add_line(mdl, 'Integrator/1',         'A_log/1',             'autorouting', 'on');
add_line(mdl, 'Stimulation S/1',      'Scope S and A/1',     'autorouting', 'on');
add_line(mdl, 'Stimulation S/1',      'S_log/1',             'autorouting', 'on');

% Solver settings
set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', ...
    'MaxStep', '1e-3', 'RelTol', '1e-6', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

%% Simulate
out = sim(mdl);
tS = out.S_log.Time;  S = squeeze(out.S_log.Data);
tA = out.A_log.Time;  A = squeeze(out.A_log.Data);

%% Plot
figure('Name', 'S and A', 'Position', [100 100 800 380]);
plot(tS, S, 'k--', 'LineWidth', 1.5); hold on;
plot(tA, A, 'LineWidth', 2);
grid on; ylim([-0.05 1.1]);
xlabel('time [s]'); ylabel('[ ]');
legend({'stimulation S', 'activation A'}, 'Location', 'northeast');
title('Excitation-contraction coupling (\tau_R = 30 ms, \tau_F = 80 ms)');

%% Key numbers for the discussion
[Amax, iMax] = max(A .* (tA < 0.6));
mask = tA > 0.5;  [Amin, iMin] = min(A(mask));  tMin = tA(mask);
fprintf('A peak  = %.3f at t = %.3f s (S peaks at 0.25 s -> lag %.0f ms)\n', ...
    Amax, tA(iMax), 1000*(tA(iMax) - 0.25));
fprintf('A trough = %.3f at t = %.3f s (S min at 0.75 s -> lag %.0f ms)\n', ...
    Amin, tMin(iMin), 1000*(tMin(iMin) - 0.75));

%% Export figure for the Overleaf write-up
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p1_2_SA.png', 'Resolution', 300);
```

Problem 1.3: build_CE.m

```
function build_CE(sys)
% BUILD_CE  Add a contractile-element (CE) subsystem to a Simulink model.
%
%   build_CE('model/CE') creates a subsystem at that path with
%     inputs : 1) S   [ ]    neural stimulation
%              2) lCE [m]    CE length
%              3) vCE [m/s]  CE velocity (<0 shortening, >0 lengthening)
%     outputs: 1) F_CE [N]   total CE force = Fmax * fl(lCE) * fv(vCE) * A
%              2) A    [ ]   muscle activation
%
%   Inside: the excitation-contraction coupling from Problem 1.2 and the
%   force-length / force-velocity relations from Problem 1.1, which are
%   evaluated with the lecture functions CE_force_length_relation.m and
%   CE_force_velocity_relation.m (must be on the path).
%
%   The block reads the parameter struct "mp" from the base workspace:
%     mp.Fmax, mp.lopt, mp.vmax, mp.w, mp.c, mp.N, mp.K, mp.tauR, mp.tauF, mp.A0

```

```
add_block('simulink/Ports & Subsystems/Subsystem', sys);
delete_line(sys, 'In1/1', 'Out1/1');
delete_block([sys '/In1']); delete_block([sys '/Out1']);

% ---- ports -----------------------------------------------------
add_block('simulink/Sources/In1',  [sys '/S'],    'Port', '1', 'Position', [30  60  60  74]);
add_block('simulink/Sources/In1',  [sys '/lCE'],  'Port', '2', 'Position', [30 200  60 214]);
add_block('simulink/Sources/In1',  [sys '/vCE'],  'Port', '3', 'Position', [30 300  60 314]);
add_block('simulink/Sinks/Out1',   [sys '/F_CE'], 'Port', '1', 'Position', [720 160 750 174]);
add_block('simulink/Sinks/Out1',   [sys '/A'],    'Port', '2', 'Position', [720  60 750  74]);

% ---- excitation-contraction coupling (Problem 1.2) -----------------
add_block('simulink/Math Operations/Sum', [sys '/S - A'], 'Inputs', '+-', 'Position', [120 55 150
    85]);
add_block('simulink/Sources/Constant', [sys '/1 over tauR'], 'Value', '1/mp.tauR', 'Position', [120
     0 170 20]);
add_block('simulink/Sources/Constant', [sys '/1 over tauF'], 'Value', '1/mp.tauF', 'Position', [120
     120 170 140]);
add_block('simulink/Signal Routing/Switch', [sys '/rising or falling'], ...
    'Criteria', 'u2 > Threshold', 'Threshold', '0', 'Position', [220 30 260 110]);
add_block('simulink/Math Operations/Product', [sys '/dA_dt'], 'Inputs', '2', 'Position', [310 50
    340 110]);
add_block('simulink/Continuous/Integrator', [sys '/Integrator'], 'InitialCondition', 'mp.A0', '
    Position', [390 65 420 95]);

add_line(sys, 'S/1',                'S - A/1',            'autorouting', 'on');
add_line(sys, 'Integrator/1',       'S - A/2',            'autorouting', 'on');
add_line(sys, 'S - A/1',            'rising or falling/2', 'autorouting', 'on');
add_line(sys, 'S - A/1',            'dA_dt/1',            'autorouting', 'on');
add_line(sys, '1 over tauR/1',      'rising or falling/1', 'autorouting', 'on');
add_line(sys, '1 over tauF/1',      'rising or falling/3', 'autorouting', 'on');
add_line(sys, 'rising or falling/1', 'dA_dt/2',           'autorouting', 'on');
add_line(sys, 'dA_dt/1',            'Integrator/1',       'autorouting', 'on');
add_line(sys, 'Integrator/1',       'A/1',                'autorouting', 'on');

% ---- force-length and force-velocity relations (Problem 1.1) -------
add_block('simulink/Math Operations/Gain', [sys '/1 over lopt'], 'Gain', '1/mp.lopt', 'Position',
    [120 192 160 222]);
add_block('simulink/Math Operations/Gain', [sys '/1 over vmax'], 'Gain', '1/mp.vmax', 'Position',
    [120 292 160 322]);
add_matlab_fcn([sys '/fl'], 'function fl = fcn(lce_nrm, mp)\nfl = CE_force_length_relation(lce_nrm,
     mp);', [220 185 300 229]);
add_matlab_fcn([sys '/fv'], 'function fv = fcn(vce_nrm, mp)\nfv = CE_force_velocity_relation(
    vce_nrm, mp);', [220 285 300 329]);

add_line(sys, 'lCE/1',         '1 over lopt/1', 'autorouting', 'on');
add_line(sys, 'vCE/1',         '1 over vmax/1', 'autorouting', 'on');
add_line(sys, '1 over lopt/1', 'fl/1',          'autorouting', 'on');
add_line(sys, '1 over vmax/1', 'fv/1',          'autorouting', 'on');

% ---- F_CE = Fmax * fl * fv * A -------------------------------------
add_block('simulink/Math Operations/Product', [sys '/A fl fv'], 'Inputs', '3', 'Position', [500 140
     530 200]);
add_block('simulink/Math Operations/Gain', [sys '/Fmax'], 'Gain', 'mp.Fmax', 'Position', [600 152
    640 182]);

add_line(sys, 'Integrator/1', 'A fl fv/1', 'autorouting', 'on');
add_line(sys, 'fl/1',         'A fl fv/2', 'autorouting', 'on');
add_line(sys, 'fv/1',         'A fl fv/3', 'autorouting', 'on');
```

```
add_line(sys, 'A fl fv/1',    'Fmax/1',    'autorouting', 'on');
add_line(sys, 'Fmax/1',       'F_CE/1',    'autorouting', 'on');
end

% ------------------------------------------------------------------------
function add_matlab_fcn(path, script, pos)
% Add a MATLAB Function block whose second argument "mp" is a parameter
% (read from the workspace, no input port), like the lecture model m01.
add_block('simulink/User-Defined Functions/MATLAB Function', path, 'Position', pos);
chart = find(sfroot, '-isa', 'Stateflow.EMChart', 'Path', path);
chart.Script = sprintf(script);
d = find(chart, '-isa', 'Stateflow.Data', 'Name', 'mp');
if isempty(d)
    d = Stateflow.Data(chart); d.Name = 'mp';
end
d.Scope = 'Parameter';
end
```

Problem 1.3: p1_3_ce.m

```
%% Problem 1.3 - Complete contractile element (CE) model in Simulink
% Builds p1_3_ce.slx: a CE subsystem (see build_CE.m) with
%     F_CE = Fmax * fl(lCE) * fv(vCE) * A
% and a small test harness: S from Problem 1.2, lCE and vCE held constant.
%
% Run as a script from the muscle_model folder (needs build_CE.m and the
% lecture functions CE_force_length_relation.m / CE_force_velocity_relation.m).

clear; close all; clc;

%% Parameters (read by the Simulink blocks from the base workspace)
mp.Fmax = 6000;          % [N]   max isometric force
mp.lopt = 0.08;          % [m]   optimum CE length
mp.vmax = 12 * mp.lopt;  % [m/s] max contraction velocity
mp.w    = 0.56;          % [lopt] force-length width
mp.c    = 0.05;          % [ ]   residual force at lopt*(1 +/- w)
mp.N    = 1.5;           % [ ]   eccentric force enhancement
mp.K    = 5;             % [ ]   force-velocity curvature
mp.tauR = 0.030;         % [s]   activation time constant
mp.tauF = 0.080;         % [s]   deactivation time constant
mp.A0   = 0;             % [ ]   initial activation
Tsim    = 1;             % [s]

%% Build the model
mdl = 'p1_3_ce';
if bdIsLoaded(mdl), close_system(mdl, 0); end
if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
new_system(mdl); open_system(mdl);

add_block('simulink/Sources/Sine Wave', [mdl '/Stimulation S'], 'SineType', 'Time based', ...
    'Amplitude', '0.5', 'Bias', '0.5', 'Frequency', '2*pi', 'Position', [40 40 80 80]);
add_block('simulink/Sources/Constant', [mdl '/lCE'], 'Value', 'lce_test', 'Position', [40 120 80
    150]);
add_block('simulink/Sources/Constant', [mdl '/vCE'], 'Value', 'vce_test', 'Position', [40 190 80
    220]);

build_CE([mdl '/CE']);
```

```
set_param([mdl '/CE'], 'Position', [180 40 300 220]);

add_block('simulink/Sinks/To Workspace', [mdl '/F_log'], 'VariableName', 'F_log', 'SaveFormat', '
    Timeseries', 'Position', [400 60 460 80]);
add_block('simulink/Sinks/To Workspace', [mdl '/A_log'], 'VariableName', 'A_log', 'SaveFormat', '
    Timeseries', 'Position', [400 150 460 170]);
add_block('simulink/Sinks/Scope', [mdl '/Scope'], 'Position', [400 220 430 250]);

add_line(mdl, 'Stimulation S/1', 'CE/1', 'autorouting', 'on');
add_line(mdl, 'lCE/1',           'CE/2', 'autorouting', 'on');
add_line(mdl, 'vCE/1',           'CE/3', 'autorouting', 'on');
add_line(mdl, 'CE/1', 'F_log/1', 'autorouting', 'on');
add_line(mdl, 'CE/1', 'Scope/1', 'autorouting', 'on');
add_line(mdl, 'CE/2', 'A_log/1', 'autorouting', 'on');

set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', 'MaxStep', '1e-3', ...
    'RelTol', '1e-6', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

%% Test 1: isometric at optimum length  ->  F_CE should equal Fmax * A
lce_test = mp.lopt;  vce_test = 0;
out1 = sim(mdl);
t  = out1.F_log.Time;  F1 = squeeze(out1.F_log.Data);  A1 = squeeze(out1.A_log.Data);

%% Test 2: longer than optimum and shortening  ->  F_CE scaled by fl*fv
lce_test = 1.2 * mp.lopt;  vce_test = -0.2 * mp.vmax;
out2 = sim(mdl);
F2 = squeeze(out2.F_log.Data);

%% Plot
figure('Name', 'CE force', 'Position', [100 100 900 350]);
subplot(1,2,1);
plot(t, 0.5*sin(2*pi*t) + 0.5, 'k--', 'LineWidth', 1.3); hold on;
plot(t, A1, 'LineWidth', 2); grid on;
xlabel('time [s]'); ylabel('[ ]'); legend({'S', 'A'}); title('Input and activation');

subplot(1,2,2);
plot(t, F1, 'LineWidth', 2); hold on; plot(t, F2, 'LineWidth', 2);
yline(mp.Fmax, ':k'); grid on; ylim([0 6600]);
xlabel('time [s]'); ylabel('F_{CE} [N]'); title('CE force');
legend({'isometric at l_{opt}', 'l_{CE} = 1.2 l_{opt}, v_{CE} = -0.2 v_{max}'}, 'Location', '
    northeast');

%% Checks
M2 = CE_force_length_relation(1.2, mp) * CE_force_velocity_relation(-0.2, mp);
fprintf('Test 1: max F_CE = %.0f N  (expected Fmax * max A = %.0f N)\n', max(F1), mp.Fmax * max(A1)
    );
fprintf('Test 2: fl*fv = %.3f, max F_CE = %.0f N  (expected %.0f N)\n', M2, max(F2), mp.Fmax * M2 *
     max(A1));

%% Export figure for the Overleaf write-up
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p1_3_FCE.png', 'Resolution', 300);
```

Problem 1.4: `run_quick_release.m`

```
function [t, lm, Fm, vm, A] = run_quick_release(m, tq, Tsim)
```

```
% RUN_QUICK_RELEASE  Simulate the quick-release experiment of Fig. 1.
%
%   [t, lm, Fm, vm, A] = run_quick_release(m, tq, Tsim)
%
%   A CE (build_CE.m, lm = lCE) hangs from the ceiling with a mass m below.
%   For t < tq the muscle is locked at lopt and fully stimulated (S = 1);
%   at t = tq the lock is released and the muscle pulls against the mass:
%
%       m * d2lm/dt2 = m*g - Fm          (lm measured downward, Fig. 1)
%
%   Returns time, muscle length [m], muscle force [N], muscle velocity [m/s]
%   (vm > 0 lengthening) and activation.
%   Reads the parameter struct "mp" from the base workspace (see p1_4_quick_release.m).

mdl = 'p1_4_quick_release';
if bdIsLoaded(mdl), close_system(mdl, 0); end
if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
new_system(mdl); open_system(mdl);

% Parameters used by the blocks (base workspace)
assignin('base', 'm_load', m);
assignin('base', 'tq', tq);

% --- muscle ------------------------------------------------------------
add_block('simulink/Sources/Constant', [mdl '/S = 1'], 'Value', '1', 'Position', [40 40 70 70]);
build_CE([mdl '/CE']);
set_param([mdl '/CE'], 'Position', [160 30 280 170]);

% --- mass dynamics:  acc = g - Fm/m  (only after release) --------------
add_block('simulink/Math Operations/Gain', [mdl '/1 over m'], 'Gain', '1/m_load', 'Position', [340
    40 380 70]);
add_block('simulink/Sources/Constant', [mdl '/g'], 'Value', '9.81', 'Position', [340 100 380 130]);
add_block('simulink/Math Operations/Sum', [mdl '/g - F over m'], 'Inputs', '-+', 'Position', [430
    60 460 90]);
add_block('simulink/Sources/Clock', [mdl '/Clock'], 'Position', [430 150 460 180]);
add_block('simulink/Sources/Constant', [mdl '/zero'], 'Value', '0', 'Position', [430 220 460 250]);
add_block('simulink/Signal Routing/Switch', [mdl '/lock'], 'Criteria', 'u2 > Threshold', ...
    'Threshold', 'tq', 'Position', [520 120 560 200]);
add_block('simulink/Continuous/Integrator', [mdl '/vm'], 'InitialCondition', '0', 'Position', [620
    145 650 175]);
add_block('simulink/Continuous/Integrator', [mdl '/lm'], 'InitialCondition', 'mp.lopt', 'Position',
     [700 145 730 175]);

add_line(mdl, 'S = 1/1',        'CE/1',            'autorouting', 'on');
add_line(mdl, 'CE/1',           '1 over m/1',      'autorouting', 'on');
add_line(mdl, '1 over m/1',     'g - F over m/1',  'autorouting', 'on');
add_line(mdl, 'g/1',            'g - F over m/2',  'autorouting', 'on');
add_line(mdl, 'g - F over m/1', 'lock/1',          'autorouting', 'on');  % after release
add_line(mdl, 'Clock/1',        'lock/2',          'autorouting', 'on');
add_line(mdl, 'zero/1',         'lock/3',          'autorouting', 'on');  % locked: acc = 0
add_line(mdl, 'lock/1',         'vm/1',            'autorouting', 'on');
add_line(mdl, 'vm/1',           'lm/1',            'autorouting', 'on');
add_line(mdl, 'lm/1',           'CE/2',            'autorouting', 'on');  % lCE = lm
add_line(mdl, 'vm/1',           'CE/3',            'autorouting', 'on');  % vCE = vm

% --- logging -----------------------------------------------------------
logs = {'Fm_log', 'CE/1'; 'A_log', 'CE/2'; 'vm_log', 'vm/1'; 'lm_log', 'lm/1'};
for k = 1:size(logs, 1)
    add_block('simulink/Sinks/To Workspace', [mdl '/' logs{k,1}], 'VariableName', logs{k,1}, ...
```

```
        'SaveFormat', 'Timeseries', 'Position', [800 40+60*k 860 60+60*k]);
    add_line(mdl, logs{k,2}, [logs{k,1} '/1'], 'autorouting', 'on');
end

set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', 'MaxStep', '2e-4', ...
    'RelTol', '1e-7', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

out = sim(mdl);
t  = out.lm_log.Time;
lm = squeeze(out.lm_log.Data);
Fm = squeeze(out.Fm_log.Data);
vm = squeeze(out.vm_log.Data);
A  = squeeze(out.A_log.Data);
end
```

Problem 1.4: p1_4_quick_release.m

```
%% Problem 1.4 - Quick-release experiment with m = 400 kg
% Muscle locked at lopt with S = 1 from t = 0; released at tq = 1 s against
% a hanging mass. Plots lm(t) and Fm(t).  Uses run_quick_release.m + build_CE.m.
% Run as a script from the muscle_model folder.

clear; close all; clc;

%% Parameters (read by the Simulink blocks from the base workspace)
mp.Fmax = 6000;  mp.lopt = 0.08;  mp.vmax = 12 * mp.lopt;
mp.w = 0.56;  mp.c = 0.05;  mp.N = 1.5;  mp.K = 5;
mp.tauR = 0.030;  mp.tauF = 0.080;  mp.A0 = 0;

m    = 400;   % [kg] hanging mass
tq   = 1;     % [s]  release time
Tsim = 2;     % [s]

%% Simulate
[t, lm, Fm, vm, A] = run_quick_release(m, tq, Tsim);

%% Plot: full experiment (left) and zoom at release (right)
figure('Name', 'Quick release', 'Position', [100 100 900 600]);
win = {[0 Tsim], [tq-0.02 tq+0.2]};
for k = 1:2
    subplot(2,2,k);
    plot(t, 100*lm, 'LineWidth', 2); hold on; grid on;
    yline(100*mp.lopt, ':k'); xline(tq, '--k');
    xlim(win{k}); ylabel('l_m [cm]');
    if k == 1, title('Full experiment'); else, title('Zoom at release'); ylim([7 8.1]); end

    subplot(2,2,k+2);
    plot(t, Fm, 'r', 'LineWidth', 2); hold on; grid on;
    yline(m*9.81, ':k', 'mg'); xline(tq, '--k');
    xlim(win{k}); xlabel('time [s]'); ylabel('F_m [N]');
    if k == 2, ylim([3700 6100]); end
end

%% Key numbers
after = t >= tq;
[vmin, i] = min(vm);
```

```
fprintf('Force before release        : %.0f N (Fmax = %.0f N)\n', Fm(find(t < tq, 1, 'last')), mp.
    Fmax);
fprintf('Weight mg                   : %.0f N\n', m*9.81);
fprintf('Peak shortening velocity    : %.3f m/s (%.2f vmax) at %.0f ms after release\n', ...
        -vmin, -vmin/mp.vmax, 1000*(t(i)-tq));
fprintf('Final length                : %.2f cm = %.2f lopt\n', 100*lm(end), lm(end)/mp.lopt);
fprintf('Time for F to reach mg+5%%  : %.0f ms\n', 1000*(t(find(after & Fm < 1.05*m*9.81, 1)) - tq)
    );

%% Export figure for the Overleaf write-up
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p1_4_qr.png', 'Resolution', 300);
```

Problem 1.5: p1_5_mass_sweep.m

```
%% Problem 1.5 - Quick release for several masses: slope vs. force-velocity relation
% Repeats the experiment of 1.4 for m = 1 ... 800 kg, measures the slope of
% lm(t) right after release (the extremum / plateau of vm), and compares
% m-vs-slope with the force-velocity relation used in the model.
% Uses run_quick_release.m + build_CE.m.  Run from the muscle_model folder.

clear; close all; clc;

%% Parameters (read by the Simulink blocks from the base workspace)
mp.Fmax = 6000;  mp.lopt = 0.08;  mp.vmax = 12 * mp.lopt;
mp.w = 0.56;  mp.c = 0.05;  mp.N = 1.5;  mp.K = 5;
mp.tauR = 0.030;  mp.tauF = 0.080;  mp.A0 = 0;

g      = 9.81;
masses = [1 10 50 100 200 400 600 700 800];   % [kg]
tq     = 1;                                   % [s]
Tsim   = 1.3;                                 % [s]  only need ~0.3 s after release
twin   = 0.15;                                % [s]  window after release to look for the slope

%% Run the sweep
nM    = numel(masses);
slope = zeros(1, nM);  tslope = zeros(1, nM);  res = cell(1, nM);
cols  = parula(nM + 1);

figure('Name', 'Velocity after release', 'Position', [100 100 1000 420]);
subplot(1,2,1); hold on; grid on;
for k = 1:nM
    m = masses(k);
    [t, lm, Fm, vm] = run_quick_release(m, tq, Tsim);
    res{k} = struct('t', t, 'lm', lm, 'Fm', Fm, 'vm', vm);

    % slope of lm right after tq = velocity right after the initial transient
    win = (t > tq + 0.002) & (t < tq + twin);
    if m * g <= mp.Fmax            % muscle lifts the mass: take the velocity extremum
        [~, i] = max(abs(vm .* win));
    else                           % mass wins: take the plateau (smallest |acceleration|)
        acc = gradient(vm, t);  acc(~win) = inf;
        [~, i] = min(abs(acc));
    end
    slope(k) = vm(i);  tslope(k) = t(i) - tq;

```

```
    plot(1000*(t - tq), vm, 'Color', cols(k,:), 'LineWidth', 1.8, 'DisplayName', sprintf('%g kg', m
    ));
    plot(1000*tslope(k), slope(k), 'o', 'MarkerFaceColor', cols(k,:), 'MarkerEdgeColor', 'k', '
    HandleVisibility', 'off');
end
yline(0, 'k', 'HandleVisibility', 'off');
xlim([-10 1000*twin]); ylim([-1 0.12]);
xlabel('time after release [ms]'); ylabel('v_m = dl_m/dt [m/s]');
title('Muscle velocity after release'); legend('Location', 'southeast', 'NumColumns', 2);

%% Mass vs. slope, compared with the force-velocity relation
% Right after release, at l = lopt and A = 1:  m g = Fmax * fv(v)  ->  m(v) = Fmax fv(v) / g
v_curve = linspace(-mp.vmax, 0.15, 500);
fv_curve = arrayfun(@(v) CE_force_velocity_relation(v / mp.vmax, mp), v_curve);
m_curve  = mp.Fmax * fv_curve / g;

subplot(1,2,2); hold on; grid on;
plot(v_curve, m_curve, 'k', 'LineWidth', 1.5, 'DisplayName', 'm = F_{max} f_v(v) / g');
plot(slope, masses, 'o', 'MarkerSize', 7, 'MarkerFaceColor', [0.85 0.2 0.2], 'MarkerEdgeColor', 'k'
    , ...
     'DisplayName', 'simulation: m vs. slope');
xline(0, ':', 'Color', [.5 .5 .5], 'HandleVisibility', 'off');
yline(mp.Fmax/g, '--', 'F_{max}/g', 'Color', [.5 .5 .5], 'HandleVisibility', 'off');
xlim([-1 0.1]); ylim([0 900]);
xlabel('slope dl_m/dt right after t_q [m/s]  (<0 shortening)'); ylabel('m [kg]');
title('Mass vs. initial slope'); legend('Location', 'northwest');

%% Table: measured slope vs. velocity predicted from fv alone
fprintf('   m [kg]   slope [m/s]   v/vmax   t [ms]   v predicted by fv [m/s]\n');
for k = 1:nM
    fv_t = masses(k) * g / mp.Fmax;                       % required fv at lopt, A = 1
    v_pred = CE_inverse_force_velocity_relation(fv_t, mp) * mp.vmax;
    fprintf('  %5d     %+.4f     %+.3f    %5.1f      %+.4f\n', masses(k), slope(k), slope(k)/mp.
    vmax, 1000*tslope(k), v_pred);
end

%% Export figure for the Overleaf write-up
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p1_5_sweep.png', 'Resolution', 300);
```

Problem 1.6: build_MTU.m

```
function build_MTU(sys)
% BUILD_MTU  Add a muscle-tendon unit (CE in series with SE) subsystem.
%
%   build_MTU('model/MTU') creates a subsystem at that path with
%     inputs : 1) S  [ ]   neural stimulation
%              2) lm [m]   total muscle length = lCE + lSE
%     outputs: 1) Fm  [N]  muscle force (= SE force = CE force)
%              2) lCE [m]  contractile element length
%              3) lSE [m]  series element length
%              4) A   [ ]  activation
%
%   Structure (same as the lecture model m01): the SE sets the force from
%   its stretch, lSE = lm - lCE; the CE must carry the same force, so its
%   velocity follows from the inverted force-velocity relation
%       fv = Fm / (Fmax fl(lCE) A)  ->  vCE = vmax * fv^-1(fv),
```

```
%   and lCE is obtained by integrating vCE.
%
%   Needs the lecture functions CE_force_length_relation.m,
%   CE_inverse_force_velocity_relation.m and SE_force_length_relation.m.
%   Reads "mp" from the base workspace: Fmax, lopt, vmax, w, c, N, K,
%   lslack, eref, tauR, tauF, A0, lce0.

add_block('simulink/Ports & Subsystems/Subsystem', sys);
delete_line(sys, 'In1/1', 'Out1/1');
delete_block([sys '/In1']); delete_block([sys '/Out1']);

% ---- ports -----------------------------------------------------------
add_block('simulink/Sources/In1', [sys '/S'],   'Port', '1', 'Position', [30  60  60  74]);
add_block('simulink/Sources/In1', [sys '/lm'],  'Port', '2', 'Position', [30 300  60 314]);
add_block('simulink/Sinks/Out1',  [sys '/Fm'],  'Port', '1', 'Position', [820 300 850 314]);
add_block('simulink/Sinks/Out1',  [sys '/lCE'], 'Port', '2', 'Position', [820 420 850 434]);
add_block('simulink/Sinks/Out1',  [sys '/lSE'], 'Port', '3', 'Position', [820 360 850 374]);
add_block('simulink/Sinks/Out1',  [sys '/A'],   'Port', '4', 'Position', [820  60 850  74]);

% ---- excitation-contraction coupling (Problem 1.2) -----------------
add_block('simulink/Math Operations/Sum', [sys '/S - A'], 'Inputs', '+-', 'Position', [120 55 150
    85]);
add_block('simulink/Sources/Constant', [sys '/1 over tauR'], 'Value', '1/mp.tauR', 'Position', [120
     0 170 20]);
add_block('simulink/Sources/Constant', [sys '/1 over tauF'], 'Value', '1/mp.tauF', 'Position', [120
     120 170 140]);
add_block('simulink/Signal Routing/Switch', [sys '/rising or falling'], ...
    'Criteria', 'u2 > Threshold', 'Threshold', '0', 'Position', [220 30 260 110]);
add_block('simulink/Math Operations/Product', [sys '/dA_dt'], 'Inputs', '2', 'Position', [310 50
    340 110]);
add_block('simulink/Continuous/Integrator', [sys '/Integrator A'], 'InitialCondition', 'mp.A0', '
    Position', [390 65 420 95]);
add_line(sys, 'S/1', 'S - A/1', 'autorouting', 'on');
add_line(sys, 'Integrator A/1', 'S - A/2', 'autorouting', 'on');
add_line(sys, 'S - A/1', 'rising or falling/2', 'autorouting', 'on');
add_line(sys, 'S - A/1', 'dA_dt/1', 'autorouting', 'on');
add_line(sys, '1 over tauR/1', 'rising or falling/1', 'autorouting', 'on');
add_line(sys, '1 over tauF/1', 'rising or falling/3', 'autorouting', 'on');
add_line(sys, 'rising or falling/1', 'dA_dt/2', 'autorouting', 'on');
add_line(sys, 'dA_dt/1', 'Integrator A/1', 'autorouting', 'on');
add_line(sys, 'Integrator A/1', 'A/1', 'autorouting', 'on');

% ---- series element: Fm = Fmax * fse(lSE / lslack) ------------------
add_block('simulink/Math Operations/Sum', [sys '/lm - lCE'], 'Inputs', '+-', 'Position', [120 295
    150 325]);
add_block('simulink/Math Operations/Gain', [sys '/1 over lslack'], 'Gain', '1/mp.lslack', 'Position
    ', [200 295 240 325]);
add_matlab_fcn([sys '/fse'], 'function fse = fcn(lse_nrm, mp)\nfse = SE_force_length_relation(
    lse_nrm, mp);', [300 288 380 332]);
add_block('simulink/Math Operations/Gain', [sys '/Fmax'], 'Gain', 'mp.Fmax', 'Position', [430 295
    470 325]);
add_line(sys, 'lm/1', 'lm - lCE/1', 'autorouting', 'on');
add_line(sys, 'lm - lCE/1', '1 over lslack/1', 'autorouting', 'on');
add_line(sys, 'lm - lCE/1', 'lSE/1', 'autorouting', 'on');
add_line(sys, '1 over lslack/1', 'fse/1', 'autorouting', 'on');
add_line(sys, 'fse/1', 'Fmax/1', 'autorouting', 'on');
add_line(sys, 'Fmax/1', 'Fm/1', 'autorouting', 'on');

% ---- CE: force-length, then invert force-velocity to get vCE ---------
```

```
add_block('simulink/Math Operations/Gain', [sys '/1 over lopt'], 'Gain', '1/mp.lopt', 'Position', [120 420 160 450]);
add_matlab_fcn([sys '/fl'], 'function fl = fcn(lce_nrm, mp)\nfl = CE_force_length_relation(lce_nrm, mp);', [200 413 280 457]);
add_block('simulink/Math Operations/Product', [sys '/A fl'], 'Inputs', '2', 'Position', [330 400 360 460]);
add_block('simulink/Math Operations/Gain', [sys '/Fmax CE'], 'Gain', 'mp.Fmax', 'Position', [400 415 440 445]);
add_block('simulink/Sources/Constant', [sys '/eps'], 'Value', 'eps', 'Position', [400 480 440 500]);
add_block('simulink/Math Operations/Sum', [sys '/no div by 0'], 'Inputs', '++', 'Position', [480 415 510 445]);
add_block('simulink/Math Operations/Product', [sys '/fv = Fm over (Fmax fl A)'], 'Inputs', '*/', 'Position', [560 390 590 450]);
add_matlab_fcn([sys '/inverse fv'], 'function vce_nrm = fcn(fv, mp)\nvce_nrm = CE_inverse_force_velocity_relation(fv, mp);', [630 398 710 442]);
add_block('simulink/Math Operations/Gain', [sys '/vmax'], 'Gain', 'mp.vmax', 'Position', [630 480 670 510]);
add_block('simulink/Continuous/Integrator', [sys '/Integrator lCE'], 'InitialCondition', 'mp.lce0', 'Position', [720 480 750 510]);
add_line(sys, 'Integrator lCE/1', '1 over lopt/1', 'autorouting', 'on');
add_line(sys, '1 over lopt/1', 'fl/1', 'autorouting', 'on');
add_line(sys, 'Integrator A/1', 'A fl/1', 'autorouting', 'on');
add_line(sys, 'fl/1', 'A fl/2', 'autorouting', 'on');
add_line(sys, 'A fl/1', 'Fmax CE/1', 'autorouting', 'on');
add_line(sys, 'Fmax CE/1', 'no div by 0/1', 'autorouting', 'on');
add_line(sys, 'eps/1', 'no div by 0/2', 'autorouting', 'on');
add_line(sys, 'Fmax/1', 'fv = Fm over (Fmax fl A)/1', 'autorouting', 'on');
add_line(sys, 'no div by 0/1', 'fv = Fm over (Fmax fl A)/2', 'autorouting', 'on');
add_line(sys, 'fv = Fm over (Fmax fl A)/1', 'inverse fv/1', 'autorouting', 'on');
add_line(sys, 'inverse fv/1', 'vmax/1', 'autorouting', 'on');
add_line(sys, 'vmax/1', 'Integrator lCE/1', 'autorouting', 'on');
add_line(sys, 'Integrator lCE/1', 'lm - lCE/2', 'autorouting', 'on');
add_line(sys, 'Integrator lCE/1', 'lCE/1', 'autorouting', 'on');
end

% ------------------------------------------------------------------------
function add_matlab_fcn(path, script, pos)
% MATLAB Function block whose second argument "mp" is a parameter (as in m01).
add_block('simulink/User-Defined Functions/MATLAB Function', path, 'Position', pos);
chart = find(sfroot, '-isa', 'Stateflow.EMChart', 'Path', path);
chart.Script = sprintf(script);
d = find(chart, '-isa', 'Stateflow.Data', 'Name', 'mp');
if isempty(d)
    d = Stateflow.Data(chart); d.Name = 'mp';
end
d.Scope = 'Parameter';
end
```

Problem 1.6: run_quick_release_SE.m

```
function [t, lm, Fm, vm, lce, A] = run_quick_release_SE(m, tq, Tsim)
% RUN_QUICK_RELEASE_SE  Quick-release experiment with a CE + SE muscle.
%
%   [t, lm, Fm, vm, lce, A] = run_quick_release_SE(m, tq, Tsim)
%
%   Same experiment as run_quick_release.m, but the muscle is the
```

```
%   muscle-tendon unit of build_MTU.m, so lm = lCE + lSE. The muscle is
%   locked at lm = lopt + lslack (SE just slack, CE at lopt) for t < tq.
%   Reads the parameter struct "mp" from the base workspace.

mdl = 'p1_6_quick_release_SE';
if bdIsLoaded(mdl), close_system(mdl, 0); end
if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
new_system(mdl); open_system(mdl);

assignin('base', 'm_load', m);
assignin('base', 'tq', tq);

% --- muscle-tendon unit ------------------------------------------------
add_block('simulink/Sources/Constant', [mdl '/S = 1'], 'Value', '1', 'Position', [40 40 70 70]);
build_MTU([mdl '/MTU']);
set_param([mdl '/MTU'], 'Position', [160 30 280 170]);

% --- mass dynamics:  acc = g - Fm/m  (only after release) --------------
add_block('simulink/Math Operations/Gain', [mdl '/1 over m'], 'Gain', '1/m_load', 'Position', [340
    40 380 70]);
add_block('simulink/Sources/Constant', [mdl '/g'], 'Value', '9.81', 'Position', [340 100 380 130]);
add_block('simulink/Math Operations/Sum', [mdl '/g - F over m'], 'Inputs', '-+', 'Position', [430
    60 460 90]);
add_block('simulink/Sources/Clock', [mdl '/Clock'], 'Position', [430 150 460 180]);
add_block('simulink/Sources/Constant', [mdl '/zero'], 'Value', '0', 'Position', [430 220 460 250]);
add_block('simulink/Signal Routing/Switch', [mdl '/lock'], 'Criteria', 'u2 > Threshold', ...
    'Threshold', 'tq', 'Position', [520 120 560 200]);
add_block('simulink/Continuous/Integrator', [mdl '/vm'], 'InitialCondition', '0', 'Position', [620
    145 650 175]);
add_block('simulink/Continuous/Integrator', [mdl '/lm'], 'InitialCondition', 'mp.lopt + mp.lslack',
     'Position', [700 145 730 175]);

add_line(mdl, 'S = 1/1',        'MTU/1',           'autorouting', 'on');
add_line(mdl, 'MTU/1',          '1 over m/1',      'autorouting', 'on');
add_line(mdl, '1 over m/1',     'g - F over m/1',  'autorouting', 'on');
add_line(mdl, 'g/1',            'g - F over m/2',  'autorouting', 'on');
add_line(mdl, 'g - F over m/1', 'lock/1',          'autorouting', 'on');
add_line(mdl, 'Clock/1',        'lock/2',          'autorouting', 'on');
add_line(mdl, 'zero/1',         'lock/3',          'autorouting', 'on');
add_line(mdl, 'lock/1',         'vm/1',            'autorouting', 'on');
add_line(mdl, 'vm/1',           'lm/1',            'autorouting', 'on');
add_line(mdl, 'lm/1',           'MTU/2',           'autorouting', 'on');   % lm = lCE + lSE

% --- logging -----------------------------------------------------------
logs = {'Fm_log', 'MTU/1'; 'lce_log', 'MTU/2'; 'A_log', 'MTU/4'; 'vm_log', 'vm/1'; 'lm_log', 'lm/1'
    };
for k = 1:size(logs, 1)
    add_block('simulink/Sinks/To Workspace', [mdl '/' logs{k,1}], 'VariableName', logs{k,1}, ...
        'SaveFormat', 'Timeseries', 'Position', [800 40+60*k 860 60+60*k]);
    add_line(mdl, logs{k,2}, [logs{k,1} '/1'], 'autorouting', 'on');
end

set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', 'MaxStep', '1e-4', ...
    'RelTol', '1e-7', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

out = sim(mdl);
t   = out.lm_log.Time;
lm  = squeeze(out.lm_log.Data);
```

```
Fm  = squeeze(out.Fm_log.Data);
vm  = squeeze(out.vm_log.Data);
lce = squeeze(out.lce_log.Data);
A   = squeeze(out.A_log.Data);
end
```

Problem 1.6: p1_6_series_elasticity.m

```
%% Problem 1.6 - Quick release with series elasticity (CE + SE)
% (a) m = 400 kg: compare lm(t), Fm(t) with and without SE.
% (b) Mass sweep with SE: m vs. initial slope of lm, and of lCE, vs. fv.
% Uses run_quick_release.m (CE only) and run_quick_release_SE.m (CE + SE).
% Run from the muscle_model folder.

clear; close all; clc;

%% Parameters (read by the Simulink blocks from the base workspace)
mp.Fmax = 6000;  mp.lopt = 0.08;  mp.vmax = 12 * mp.lopt;
mp.w = 0.56;  mp.c = 0.05;  mp.N = 1.5;  mp.K = 5;
mp.tauR = 0.030;  mp.tauF = 0.080;
mp.lslack = 0.23;   % [m] SE slack length (lecture VAS value)
mp.eref   = 0.04;   % [ ] SE reference strain at Fmax (lecture value)
mp.preA   = 0.01;   % [ ] pre-activation, avoids 0/0 in the inverted fv (as in m01)
mp.lce0   = mp.lopt;

g = 9.81;  tq = 1;  Tsim = 2;

%% (a) m = 400 kg with and without SE
m = 400;
mp.A0 = 0;        [t1, lm1, F1, v1]       = run_quick_release(m, tq, Tsim);
mp.A0 = mp.preA;  [t2, lm2, F2, v2, lce2] = run_quick_release_SE(m, tq, Tsim);

figure('Name', 'Quick release with SE', 'Position', [100 100 900 600]);
win = {[0 Tsim], [tq-0.02 tq+0.3]};
for k = 1:2
    subplot(2,2,k); hold on; grid on;
    plot(t1, 100*(lm1 - mp.lopt), 'LineWidth', 2);
    plot(t2, 100*(lm2 - mp.lopt - mp.lslack), '--', 'LineWidth', 2);
    xline(tq, '--k'); xlim(win{k}); ylabel('\Delta l_m [cm]');
    if k == 1, title('Full experiment'); legend({'CE only', 'CE + SE'}, 'Location', 'southwest');
    else, title('Zoom at release'); ylim([-1.5 0.15]); end
    subplot(2,2,k+2); hold on; grid on;
    plot(t1, F1, 'LineWidth', 2); plot(t2, F2, '--', 'LineWidth', 2);
    yline(m*g, ':k', 'mg'); xline(tq, '--k'); xlim(win{k}); xlabel('time [s]'); ylabel('F_m [N]');
    if k == 2, ylim([3300 6100]); end
end
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p1_6_qr.png', 'Resolution', 300);

i0 = find(t2 < tq, 1, 'last');
fprintf('With SE, before release: Fm = %.0f N, lCE = %.3f lopt, lSE stretch = %.1f mm\n', ...
        F2(i0), lce2(i0)/mp.lopt, 1000*(lm2(i0) - lce2(i0) - mp.lslack));
fprintf('With SE, after release : min Fm = %.0f N (undershoot below mg = %.0f N), peak v_m = %.3f m
    /s\n', ...
        min(F2(t2 > tq)), m*g, min(v2));
fprintf('CE only, after release : peak v_m = %.3f m/s\n', min(v1));

```

```
%% (b) Mass sweep with SE: slope of lm and slope of lCE
masses = [1 10 50 100 200 400 600 700 800];
twin = 0.15;  nM = numel(masses);
slope_noSE = zeros(1,nM);  slope_lm = zeros(1,nM);  slope_lce = zeros(1,nM);

for k = 1:nM
    m = masses(k);  lifts = (m*g <= mp.Fmax);
    mp.A0 = 0;        [t, ~, ~, vm] = run_quick_release(m, tq, 1.35);
    win = (t > tq + 0.002) & (t < tq + twin);
    slope_noSE(k) = local_pick(vm, gradient(vm, t), t, win, lifts);

    mp.A0 = mp.preA;  [t, ~, ~, vm, lce] = run_quick_release_SE(m, tq, 1.35);
    win = (t > tq + 0.002) & (t < tq + twin);
    vce = gradient(lce, t);
    slope_lm(k)  = local_pick(vm,  gradient(vm, t),  t, win, lifts);
    slope_lce(k) = local_pick(vce, gradient(vce, t), t, win, lifts);
end

v_curve  = linspace(-mp.vmax, 0.15, 500);
fv_curve = arrayfun(@(v) CE_force_velocity_relation(v / mp.vmax, mp), v_curve);

figure('Name', 'Mass vs slope with SE', 'Position', [100 100 640 460]); hold on; grid on;
plot(v_curve, mp.Fmax * fv_curve / g, 'k', 'LineWidth', 1.5, 'DisplayName', 'm = F_{max} f_v(v) / g
    ');
plot(slope_noSE, masses, 'o', 'MarkerSize', 7, 'MarkerFaceColor', [.85 .2 .2], 'MarkerEdgeColor', '
    k', 'DisplayName', 'CE only: slope of l_m');
plot(slope_lm,   masses, 's', 'MarkerSize', 7, 'MarkerFaceColor', [.2 .6 .2], 'MarkerEdgeColor', 'k
    ', 'DisplayName', 'CE + SE: slope of l_m');
plot(slope_lce,  masses, '^', 'MarkerSize', 7, 'MarkerFaceColor', [1 .5 .1],  'MarkerEdgeColor', 'k
    ', 'DisplayName', 'CE + SE: dl_{CE}/dt only');
xline(0, ':', 'Color', [.5 .5 .5], 'HandleVisibility', 'off');
yline(mp.Fmax/g, '--', 'F_{max}/g', 'Color', [.5 .5 .5], 'HandleVisibility', 'off');
xlim([-1.02 0.12]); ylim([0 900]);
xlabel('slope right after t_q [m/s]  (<0 shortening)'); ylabel('m [kg]'); legend('Location', '
    northwest');
exportgraphics(gcf, 'figures/p1_6_sweep.png', 'Resolution', 300);

fprintf('\n   m [kg]   CE only   SE: dl_m/dt   SE: dl_CE/dt   from fv\n');
for k = 1:nM
    v_pred = CE_inverse_force_velocity_relation(masses(k)*g/mp.Fmax, mp) * mp.vmax;
    fprintf('  %5d    %+.3f     %+.3f        %+.3f       %+.3f\n', masses(k), slope_noSE(k),
    slope_lm(k), slope_lce(k), v_pred);
end

%% helper: value of v "right after release" (extremum if the muscle lifts the mass, plateau
    otherwise)
function s = local_pick(v, a, t, win, lifts)
if lifts
    [~, i] = max(abs(v .* win));
else
    a(~win) = inf;  [~, i] = min(abs(a));
end
s = v(i);
end
```

Problem 2.1: `build_neuron.m`

```
function build_neuron(sys, x_init)
% BUILD_NEURON  Add a Matsuoka neuron subsystem to a Simulink model.
%
%   build_neuron('model/e_L')         membrane potential starts at 0
%   build_neuron('model/e_L', '1')    membrane potential starts at 1
%
%   Model (Matsuoka 1985/1987, one neuron with self-adaptation):
%       y   = max(x, 0)                              firing rate (output)
%       x'  = (1/tau_x) * (x0 + sum_i w_i y_i - w' * xa - x)   membrane potential
%       xa' = (1/tau_a) * (y - xa)                   adaptation ("fatigue")
%
%   Ports:   inputs  1) x0        central (tonic) drive
%                    2) sum_wy    weighted input from other neurons
%            outputs 1) y         firing rate
%                    2) x         membrane potential
%                    3) xa        adaptation state
%
%   Each first-order ODE  tau * dz/dt = (inputs) - z  is built as
%       Sum  ->  Gain 1/tau  ->  Integrator  (with z fed back into the Sum),
%   the same pattern as the activation dynamics in Problem 1.2.
%
%   Reads the parameter struct "np" from the base workspace:
%       np.taux, np.taua, np.wp

if nargin < 2, x_init = '0'; end

add_block('simulink/Ports & Subsystems/Subsystem', sys);
delete_line(sys, 'In1/1', 'Out1/1');
delete_block([sys '/In1']); delete_block([sys '/Out1']);

% ---- ports -----------------------------------------------------------
add_block('simulink/Sources/In1', [sys '/x0'],     'Port', '1', 'Position', [30  80  60  94]);
add_block('simulink/Sources/In1', [sys '/sum_wy'], 'Port', '2', 'Position', [30 120  60 134]);
add_block('simulink/Sinks/Out1',  [sys '/y'],      'Port', '1', 'Position', [620 100 650 114]);
add_block('simulink/Sinks/Out1',  [sys '/x'],      'Port', '2', 'Position', [620 160 650 174]);
add_block('simulink/Sinks/Out1',  [sys '/xa'],     'Port', '3', 'Position', [620 260 650 274]);

% ---- membrane potential:  tau_x * dx/dt = x0 + sum_wy - w'*xa - x ------
add_block('simulink/Math Operations/Sum', [sys '/sum x'], 'Inputs', '++--', ...
    'IconShape', 'rectangular', 'Position', [140 80 170 150]);
add_block('simulink/Math Operations/Gain', [sys '/1 over taux'], 'Gain', '1/np.taux', 'Position',
    [220 100 260 130]);
add_block('simulink/Continuous/Integrator', [sys '/int x'], 'InitialCondition', x_init, 'Position',
     [310 100 340 130]);

% ---- output nonlinearity:  y = max(x, 0) -----------------------------
add_block('simulink/Discontinuities/Saturation', [sys '/max(x,0)'], ...
    'UpperLimit', 'inf', 'LowerLimit', '0', 'Position', [400 100 430 130]);

% ---- adaptation:  tau_a * dxa/dt = y - xa ----------------------------
add_block('simulink/Math Operations/Sum', [sys '/sum xa'], 'Inputs', '+-', 'Position', [400 250 430
     280]);
add_block('simulink/Math Operations/Gain', [sys '/1 over taua'], 'Gain', '1/np.taua', 'Position',
    [460 250 500 280]);
add_block('simulink/Continuous/Integrator', [sys '/int xa'], 'InitialCondition', '0', 'Position',
    [540 250 570 280]);
add_block('simulink/Math Operations/Gain', [sys '/w prime'], 'Gain', 'np.wp', ...
    'Orientation', 'left', 'Position', [220 200 260 230]);

```

```
% ---- wiring ----------------------------------------------------------------
add_line(sys, 'x0/1',          'sum x/1',       'autorouting', 'on');
add_line(sys, 'sum_wy/1',      'sum x/2',       'autorouting', 'on');
add_line(sys, 'w prime/1',     'sum x/3',       'autorouting', 'on');   % - w' xa
add_line(sys, 'int x/1',       'sum x/4',       'autorouting', 'on');   % - x   (leak)
add_line(sys, 'sum x/1',       '1 over taux/1', 'autorouting', 'on');
add_line(sys, '1 over taux/1', 'int x/1',       'autorouting', 'on');
add_line(sys, 'int x/1',       'max(x,0)/1',    'autorouting', 'on');
add_line(sys, 'int x/1',       'x/1',           'autorouting', 'on');
add_line(sys, 'max(x,0)/1',    'y/1',           'autorouting', 'on');
add_line(sys, 'max(x,0)/1',    'sum xa/1',      'autorouting', 'on');   % + y
add_line(sys, 'int xa/1',      'sum xa/2',      'autorouting', 'on');   % - xa (leak)
add_line(sys, 'sum xa/1',      '1 over taua/1', 'autorouting', 'on');
add_line(sys, '1 over taua/1', 'int xa/1',      'autorouting', 'on');
add_line(sys, 'int xa/1',      'xa/1',          'autorouting', 'on');
add_line(sys, 'int xa/1',      'w prime/1',     'autorouting', 'on');
end
```

Problem 2.1: p2_1_neuron.m

```
%% Problem 2.1 - A single Matsuoka neuron (left extensor) in Simulink
% Builds p2_1_neuron.slx around the neuron subsystem of build_neuron.m,
% simulates the baseline, then sweeps the central input x0 and the
% adaptation time constant tau_a.  Run from the folder that holds build_neuron.m.

clear; close all; clc;

%% Neuron parameters (read by the Simulink blocks from the base workspace)
np.x0   = 5;     % [ ] central (tonic) input
np.taux = 1;     % [s] membrane time constant
np.taua = 12;    % [s] adaptation time constant
np.wp   = 2.5;   % [ ] adaptation weight w'
Tsim    = 40;    % [s]

%% Build the model:  x0 --> [neuron] --> y
mdl = 'p2_1_neuron';
if bdIsLoaded(mdl), close_system(mdl, 0); end
if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
new_system(mdl); open_system(mdl);

add_block('simulink/Sources/Constant', [mdl '/x0'],     'Value', 'np.x0', 'Position', [40 60 80
    90]);
add_block('simulink/Sources/Constant', [mdl '/no input'], 'Value', '0',     'Position', [40 130 80
    160]);  % sum_i w_i y_i = 0
build_neuron([mdl '/e_L']);
set_param([mdl '/e_L'], 'Position', [160 50 260 170]);

logs = {'y_log', 'e_L/1'; 'x_log', 'e_L/2'; 'xa_log', 'e_L/3'};
for k = 1:size(logs, 1)
    add_block('simulink/Sinks/To Workspace', [mdl '/' logs{k,1}], 'VariableName', logs{k,1}, ...
        'SaveFormat', 'Timeseries', 'Position', [360 40+50*k 420 60+50*k]);
    add_line(mdl, logs{k,2}, [logs{k,1} '/1'], 'autorouting', 'on');
end
add_block('simulink/Sinks/Scope', [mdl '/Scope'], 'Position', [360 220 390 250]);
add_line(mdl, 'x0/1',       'e_L/1',   'autorouting', 'on');
add_line(mdl, 'no input/1', 'e_L/2',   'autorouting', 'on');
add_line(mdl, 'e_L/1',      'Scope/1', 'autorouting', 'on');
```

```

set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', 'MaxStep', '0.01', ...
    'RelTol', '1e-6', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

%% Helper: run the model with the current np and return [t, y, x, xa]
run_it = @() deal_sim(mdl);

%% 1) Baseline
[t, y, x, xa] = run_it();
figure('Name', 'Single neuron', 'Position', [100 100 1200 360]);
subplot(1,3,1); hold on; grid on;
plot(t, x, '--', 'Color', [.5 .5 .5], 'LineWidth', 1.5);
plot(t, xa, 'LineWidth', 1.5); plot(t, y, 'LineWidth', 2.2);
yline(np.x0 / (1 + np.wp), ':k');
xlabel('time [s]'); title(sprintf('Baseline (x_0 = %g, \\tau_a = %g)', np.x0, np.taua));
legend({'membrane x', 'adaptation x''', 'output y'}, 'Location', 'northeast');
fprintf('Baseline: y peaks at %.2f (t = %.1f s), settles at %.3f (predicted x0/(1+w'') = %.3f)\n',
    ...
        max(y), t(find(y == max(y), 1)), y(end), np.x0 / (1 + np.wp));

%% 2) Sweep central input x0
subplot(1,3,2); hold on; grid on;
for x0 = [-2 1 5 10]
    np.x0 = x0;  [t, y] = run_it();
    plot(t, y, 'LineWidth', 2, 'DisplayName', sprintf('x_0 = %g', x0));
end
np.x0 = 5;
xlabel('time [s]'); ylabel('y'); title('Effect of x_0 (\tau_a = 12)'); legend('Location', '
    northeast');

%% 3) Sweep adaptation time constant tau_a
subplot(1,3,3); hold on; grid on;
for taua = [1 3 12 50]
    np.taua = taua;  [t, y] = run_it();
    plot(t, y, 'LineWidth', 2, 'DisplayName', sprintf('\\tau_a = %g', taua));
end
np.taua = 12;
xlabel('time [s]'); ylabel('y'); title('Effect of \tau_a (x_0 = 5)'); legend('Location', 'northeast
    ');

%% Export figure for the Overleaf write-up
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p2_1_neuron.png', 'Resolution', 300);

%% ---- local function: simulate and unpack the logged signals ----------
function [t, y, x, xa] = deal_sim(mdl)
out = sim(mdl);
t  = out.y_log.Time;
y  = squeeze(out.y_log.Data);
x  = squeeze(out.x_log.Data);
xa = squeeze(out.xa_log.Data);
end
```

Problem 2.2: p2_2_oscillator.m

```

%% Problem 2.2 - Extensor-flexor pair: a Matsuoka half-center oscillator
```

```
% Two neurons from build_neuron.m inhibit each other:
%     e receives  w_fe * y_f      (w_fe = -1.5)
%     f receives  w_ef * y_e      (w_ef = -1.5)
% with x_e(0) = 1 so the symmetric pair has something to break the tie.
% Then: sweep tau_a (what happens below 2*tau_x?) and the weight imbalance.
% Run from the cpg_model folder.

clear; close all; clc;

%% Parameters (read by the Simulink blocks from the base workspace)
np.x0   = 5;     np.taux = 1;     np.taua = 12;    np.wp = 2.5;
np.wfe  = -1.5;  % weight from flexor to extensor (input to e)
np.wef  = -1.5;  % weight from extensor to flexor (input to f)
Tsim    = 100;

%% Build the model
mdl = 'p2_2_oscillator';
if bdIsLoaded(mdl), close_system(mdl, 0); end
if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
new_system(mdl); open_system(mdl);

add_block('simulink/Sources/Constant', [mdl '/x0'], 'Value', 'np.x0', 'Position', [40 120 80 150]);
build_neuron([mdl '/e_L'], '1');     % extensor, x_e(0) = 1
build_neuron([mdl '/f_L'], '0');     % flexor,   x_f(0) = 0
set_param([mdl '/e_L'], 'Position', [220 30 320 130]);
set_param([mdl '/f_L'], 'Position', [220 190 320 290]);

% synaptic weights live OUTSIDE the neurons, on the wires between them
add_block('simulink/Math Operations/Gain', [mdl '/w_fe'], 'Gain', 'np.wfe', 'Orientation', 'left',
    'Position', [400 150 440 180]);
add_block('simulink/Math Operations/Gain', [mdl '/w_ef'], 'Gain', 'np.wef', 'Orientation', 'left',
    'Position', [400 90 440 120]);

add_line(mdl, 'x0/1', 'e_L/1', 'autorouting', 'on');     % tonic drive to both
add_line(mdl, 'x0/1', 'f_L/1', 'autorouting', 'on');
add_line(mdl, 'f_L/1', 'w_fe/1', 'autorouting', 'on');   % y_f * w_fe -> e
add_line(mdl, 'w_fe/1', 'e_L/2', 'autorouting', 'on');
add_line(mdl, 'e_L/1', 'w_ef/1', 'autorouting', 'on');   % y_e * w_ef -> f
add_line(mdl, 'w_ef/1', 'f_L/2', 'autorouting', 'on');

add_block('simulink/Sinks/To Workspace', [mdl '/ye_log'], 'VariableName', 'ye_log', 'SaveFormat', '
    Timeseries', 'Position', [520 40 580 60]);
add_block('simulink/Sinks/To Workspace', [mdl '/yf_log'], 'VariableName', 'yf_log', 'SaveFormat', '
    Timeseries', 'Position', [520 260 580 280]);
add_block('simulink/Sinks/Scope', [mdl '/Scope'], 'Position', [520 140 550 170]);
set_param([mdl '/Scope'], 'NumInputPorts', '2');
add_line(mdl, 'e_L/1', 'ye_log/1', 'autorouting', 'on');
add_line(mdl, 'f_L/1', 'yf_log/1', 'autorouting', 'on');
add_line(mdl, 'e_L/1', 'Scope/1',  'autorouting', 'on');
add_line(mdl, 'f_L/1', 'Scope/2',  'autorouting', 'on');

set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', 'MaxStep', '0.01', ...
    'RelTol', '1e-6', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

%% (a) Baseline
[t, ye, yf] = run_osc(mdl);
figure('Name', 'Oscillator', 'Position', [100 100 1000 640]);
subplot(2,2,1); hold on; grid on;
```

```
plot(t, ye, 'LineWidth', 2); plot(t, yf, 'LineWidth', 2); xlim([0 60]);
xlabel('time [s]'); ylabel('output'); title('Baseline (w_{fe} = w_{ef} = -1.5)'); legend({'y_e', 'y_f'});
r = osc_stats(t, ye, 40);
fprintf('Baseline: period %.1f s, amplitude %.2f, extensor active %.0f%% of the period\n', r.period, r.amp, 100*r.duty);

%% (b) Sweep tau_a: period and amplitude; oscillation should die below 2*tau_x
taus = [1 1.5 1.8 2 2.2 2.5 3 4 6 8 12 20 30 50];
per = nan(size(taus));  amp = nan(size(taus));
for k = 1:numel(taus)
    np.taua = taus(k);
    set_param(mdl, 'StopTime', num2str(max(150, 8*taus(k))));
    [t, ye] = run_osc(mdl);
    r = osc_stats(t, ye, 0.4*t(end));
    if r.osc, per(k) = r.period; amp(k) = r.amp; end
    fprintf('tau_a = %4.1f : %s\n', taus(k), ternary(r.osc, sprintf('period %.1f s, amp %.2f', r.period, r.amp), 'no oscillation'));
end
np.taua = 12;  set_param(mdl, 'StopTime', num2str(Tsim));
subplot(2,2,2); hold on; grid on;
yyaxis left;  plot(taus, per, 'o-', 'LineWidth', 1.8); ylabel('period [s]');
yyaxis right; plot(taus, amp, 's--', 'LineWidth', 1.5); ylabel('amplitude of y_e'); ylim([0 3.5]);
set(gca, 'XScale', 'log'); xline(2, '--k', '\tau_a = 2\tau_x'); xlabel('\tau_a [s]'); title('Period vs. adaptation time');

%% (c) Time traces just above and below 2*tau_x
subplot(2,2,3); hold on; grid on;
for taua = [3 1.5]
    np.taua = taua;  [t, ye, yf] = run_osc(mdl);
    ls = ternary(taua > 2, '-', '--');
    plot(t, ye, ls, 'Color', [0 .45 .74], 'LineWidth', 2, 'DisplayName', sprintf('y_e, \\tau_a = %g', taua));
    plot(t, yf, ls, 'Color', [.85 .33 .1], 'LineWidth', 2, 'DisplayName', sprintf('y_f, \\tau_a = %g', taua));
end
np.taua = 12;
xlim([0 40]); xlabel('time [s]'); ylabel('output'); title('\tau_a above vs. below 2\tau_x'); legend('NumColumns', 2);

%% (d) Weight imbalance: extensor inhibits flexor harder
fprintf('\nImbalance (w_fe = -1.5 fixed):\n');
for wef = [-0.5 -1 -1.5 -2 -3]
    np.wef = wef;  [t, ye, yf] = run_osc(mdl);
    r = osc_stats(t, ye, 40);  m = t > 40;
    fprintf('  w_ef = %4.1f : %s, e dominant %.0f%% of the time, peaks y_e %.2f / y_f %.2f\n', wef, ...
        ternary(r.osc, sprintf('period %.1f s', r.period), 'no oscillation'), 100*mean(ye(m) > yf(m)), max(ye(m)), max(yf(m)));
    if wef == -3, te = t; yee = ye; yff = yf; end
end
np.wef = -1.5;
subplot(2,2,4); hold on; grid on;
m = te >= 30;  plot(te(m)-30, yee(m), 'LineWidth', 2); plot(te(m)-30, yff(m), 'LineWidth', 2);
xlim([0 70]); xlabel('time [s]'); ylabel('output'); title('Imbalance: w_{fe} = -1.5, w_{ef} = -3'); legend({'y_e', 'y_f'});

if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p2_2_oscillator.png', 'Resolution', 300);
```

```
106
107 %% ---- local functions ------------------------------------------------
108 function [t, ye, yf] = run_osc(mdl)
109 out = sim(mdl);
110 t = out.ye_log.Time;  ye = squeeze(out.ye_log.Data);  yf = squeeze(out.yf_log.Data);
111 end
112
113 function r = osc_stats(t, y, t0)
114 % period from successive onsets (y rising through 0), amplitude, duty cycle
115 m = t > t0;  t = t(m);  y = y(m);
116 on = y > 1e-6;  onsets = find(diff(on) == 1);
117 r.osc = numel(onsets) >= 2 && (max(y) - min(y)) > 1e-3;
118 r.amp = max(y);  r.period = NaN;  r.duty = NaN;
119 if r.osc
120     r.period = mean(diff(t(onsets)));
121     r.duty   = mean(on(onsets(1):onsets(end)));
122 end
123 end
124
125 function out = ternary(c, a, b)
126 if c, out = a; else, out = b; end
127 end
```

Problem 2.3: p2_3_gaits.m

```
1  %% Problem 2.3 - Two coupled half-center oscillators: walking, running, hopping
2  % Four neurons (eL, fL, eR, fR) from build_neuron.m, connected all-to-all as
3  % in Fig. 2 of the assignment through a 4x4 weight matrix W, where
4  %     W(i,j) = weight from neuron j INTO neuron i,   order [eL fL eR fR].
5  % Each weight is a Gain block on the wire; the neuron blocks never change.
6  % Three weight presets reproduce the extensor patterns of Fig. 2.
7  % Run from the cpg_model folder.
8
9  clear; close all; clc;
10
11 %% Neuron parameters
12 np.x0 = 5;  np.taux = 1;  np.taua = 12;  np.wp = 2.5;
13 Tsim  = 200;
14
15 names = {'eL', 'fL', 'eR', 'fR'};
16 idx   = struct('eL', 1, 'fL', 2, 'eR', 3, 'fR', 4);
17
18 %% Build the 4-neuron network once; weights are read from the matrix W
19 mdl = 'p2_3_gaits';
20 if bdIsLoaded(mdl), close_system(mdl, 0); end
21 if exist([mdl '.slx'], 'file'), delete([mdl '.slx']); end
22 new_system(mdl); open_system(mdl);
23
24 add_block('simulink/Sources/Constant', [mdl '/x0'], 'Value', 'np.x0', 'Position', [20 300 60 330]);
25 ypos = [40 200 400 560];
26 for i = 1:4
27     build_neuron([mdl '/' names{i}], sprintf('xinit(%d)', i));          % initial potential from
       vector xinit
28     set_param([mdl '/' names{i}], 'Position', [260 ypos(i) 360 ypos(i)+90]);
29     add_line(mdl, 'x0/1', [names{i} '/1'], 'autorouting', 'on');       % same tonic drive to all
30     % the input sum_i w_i y_i of neuron i: Sum of the three other outputs, each through a Gain W(i,
       j)
```

```
    add_block('simulink/Math Operations/Sum', [mdl '/sum ' names{i}], 'Inputs', '+++', ...
        'Position', [180 ypos(i)+40 200 ypos(i)+70]);
    add_line(mdl, ['sum ' names{i} '/1'], [names{i} '/2'], 'autorouting', 'on');
    add_block('simulink/Sinks/To Workspace', [mdl '/' names{i} '_log'], 'VariableName', ['y_' names{i}], ...
        'SaveFormat', 'Timeseries', 'Position', [460 ypos(i)+30 520 ypos(i)+50]);
    add_line(mdl, [names{i} '/1'], [names{i} '_log/1'], 'autorouting', 'on');
end
for i = 1:4                                   % wire  y_j --> Gain W(i,j) --> sum_i
    others = setdiff(1:4, i);
    for k = 1:3
        j = others(k);
        g = sprintf('W_%s_from_%s', names{i}, names{j});
        add_block('simulink/Math Operations/Gain', [mdl '/' g], 'Gain', sprintf('W(%d,%d)', i, j), ...
            'Orientation', 'left', 'Position', [100 ypos(i)+25*k 140 ypos(i)+25*k+20]);
        add_line(mdl, [names{j} '/1'], [g '/1'], 'autorouting', 'on');
        add_line(mdl, [g '/1'], sprintf('sum %s/%d', names{i}, k), 'autorouting', 'on');
    end
end
set_param(mdl, 'StopTime', num2str(Tsim), 'Solver', 'ode45', 'MaxStep', '0.02', ...
    'RelTol', '1e-6', 'ReturnWorkspaceOutputs', 'on');
save_system(mdl);

%% Gait presets  (w_ef: e->f within leg, w_fe: f->e within leg,
%                 w_ee: eL<->eR, w_ff: fL<->fR, w_ef_x: e -> contralateral f)
gaits(1) = struct('name', 'Walk', 'w_ef', -2.0, 'w_fe', -1.0, 'w_ee', -0.5, 'w_ff', -0.5, 'w_ef_x', 0,    'xinit', [1 0 0 0]);
gaits(2) = struct('name', 'Run',  'w_ef', -1.0, 'w_fe', -2.5, 'w_ee', -1.0, 'w_ff',  0,   'w_ef_x', 0,    'xinit', [1 0 0 0]);
gaits(3) = struct('name', 'Hop',  'w_ef', -1.0, 'w_fe', -2.5, 'w_ee',  0,   'w_ff',  0,   'w_ef_x', -0.5, 'xinit', [1 0 0 0]);

figure('Name', 'Gaits', 'Position', [60 60 1300 600]);
for g = 1:3
    G = gaits(g);
    W = zeros(4);
    W(idx.fL, idx.eL) = G.w_ef;   W(idx.eL, idx.fL) = G.w_fe;      % left leg half-center
    W(idx.fR, idx.eR) = G.w_ef;   W(idx.eR, idx.fR) = G.w_fe;      % right leg half-center
    W(idx.eR, idx.eL) = G.w_ee;   W(idx.eL, idx.eR) = G.w_ee;      % extensor <-> extensor
    W(idx.fR, idx.fL) = G.w_ff;   W(idx.fL, idx.fR) = G.w_ff;      % flexor <-> flexor
    W(idx.fR, idx.eL) = G.w_ef_x; W(idx.fL, idx.eR) = G.w_ef_x;    % extensor -> contralateral flexor
    xinit = G.xinit;                                                %#ok<NASGU>  read by the integrators

    out = sim(mdl);
    t = out.y_eL.Time;
    y = [squeeze(out.y_eL.Data) squeeze(out.y_fL.Data) squeeze(out.y_eR.Data) squeeze(out.y_fR.Data)];

    r = gait_stats(t, y, 100);
    fprintf('%-4s: period %.1f s | extensor duty %.2f | eR-eL phase %.2f | both ext off %.2f | both ext on %.2f\n', ...
        G.name, r.period, r.duty, r.phase, r.both_off, r.both_on);

    % plot three periods after the transient, traces on top and Fig.2-style bars below
    T0 = 120;  m = t >= T0 & t <= T0 + 3*r.period;  tt = t(m) - T0;
    subplot(2,3,g); hold on; grid on;
```

```
    plot(tt, y(m,3), 'r', 'LineWidth', 3);  plot(tt, y(m,1), 'b', 'LineWidth', 1.8);
    plot(tt, y(m,2), 'b:', tt, y(m,4), 'r:');
    ylim([0 4.2]); xlim([0 tt(end)]); ylabel('output');
    title(sprintf('%s: w_{ef}=%g, w_{fe}=%g, w_{ee}=%g, w_{ff}=%g, w_{ef,x}=%g', G.name, G.w_ef, G.
    w_fe, G.w_ee, G.w_ff, G.w_ef_x), 'FontSize', 9);
    if g == 1, legend({'y_{eR}', 'y_{eL}', 'y_{fL}', 'y_{fR}'}, 'Location', 'northeast'); end
    subplot(2,3,g+3); hold on; grid on;
    draw_bars(tt, y(m,3) > 1e-6, 1);  draw_bars(tt, y(m,1) > 1e-6, 0);
    yticks([0 1]); yticklabels({'eL', 'eR'}); ylim([-0.6 1.6]); xlim([0 tt(end)]); xlabel('time [s]
    ');
end
if ~exist('figures', 'dir'), mkdir('figures'); end
exportgraphics(gcf, 'figures/p2_3_gaits.png', 'Resolution', 300);

%% ---- local functions ------------------------------------------------
function r = gait_stats(t, y, t0)
% period and duty from eL onsets; phase of eR onset relative to eL; overlap measures
m = t > t0;  t = t(m);  eL = y(m,1);  eR = y(m,3);
onL = eL > 1e-6;  onR = eR > 1e-6;
oL = find(diff(onL) == 1);  oR = find(diff(onR) == 1);
r.period = mean(diff(t(oL)));  r.duty = mean(onL(oL(1):oL(end)));
tL = t(oL(end-1));  tR = t(oR(find(t(oR) >= tL, 1)));
r.phase = mod((tR - tL) / r.period, 1);
r.both_off = mean(~onL & ~onR);  r.both_on = mean(onL & onR);
end

function draw_bars(t, on, level)
d = diff([0; on(:); 0]);  s = find(d == 1);  e = find(d == -1) - 1;
for k = 1:numel(s), plot([t(s(k)) t(e(k))], [level level], 'k', 'LineWidth', 6); end
end
```